Proto-Eight Meme Engineering: A Practical Systems Playbook Built on Incubation Trigram (先天八卦)

Contents

Part 0 — Orientation & Toolkit

Ch.0 How to Use This Book (and the Twin Volume)
Ch.1 Eight Primitives Cheat-Sheet (Bāguà → Engineering)

Part I — The Four Dyads (One dyad per chapter)

Ch.2 乾×坤 **— Gradient & Gate: Two-Tank Flow Control**
Ch.3 艮×兌 **— Boundary/Buffer × Exchange: Bullwhip Taming**
Ch.4 震×巽 **— Trigger × Guidance: Nudge, Route, Convert**
Ch.5 坎×離 **— Memory × Focus: Attention as Control Surface**

Part II — Two-Dyad × Two-Dyad Modes (4 canonical patterns)

Ch.6 Ventilate–Store (艮兌 **+** 坎離**)**
Breathing cycles: exchange + memory.
Ch.7 Ignite–Guide (震巽 **+** 離**)**
Campaign ignition + path steering without churn.
Ch.8 Seal–Bleed (乾坤 **+** 艮兌**)**
Gate hard where it matters; bleed where it pays.
Ch.9 Pulse–Soak (震巽 **+** 坎**)**
Short pulses; long soak into memory.

Part III — Triads (the "compounding kits")

Ch.10 Compounding Trio: Gradient + Retention + Buffer
Ch.11 Crisis Trio: Trigger + Boundary + Memory (Firebreaks)
Ch.12 Growth Flywheel: Gate + Guide + Focus

Part IV — Four-in-One: The Eight-Node Operating Diagram

Ch.13 The Eight-Node Control Board (先天八卦 **as Ops Map)**
Ch.14 Synchronization, Drift, and Debt

Part V — Domain Playbooks (same skeleton, ready-to-run)

Ch.15 Software Delivery — feature gating, rollout buffers, incident firebreaks.
Ch.16 Supply Chain & Inventory — dampers, reorder topology, seal-bleed policy.
Ch.17 Content & Community — pulse-soak, memory resurfacing, fatigue radar.
Ch.18 Org & Finance — KPI "photons" (reports) as observables; cadence design.

Each playbook includes: ready dashboards, standard labs, pitfalls, and a one-line Ô-peek.

Part VI — The Lab Handbook

Ch.19 The 12-Period Experiment Suite
Ch.20 Metrics, Alerting, and Saturation Hygiene

Appendices

A. Bāguà ↔ Engineering Primitive Map (1-page)
B. KPI & Equation Cheats (ready to paste into notebooks)
C. Case Card Library (dozens of short, runnable scenarios)
D. Ô-peek Cross-Reference (chapter-by-chapter pointers into Book 2 topics like Ô, τ, phase alignment, semantic BH near-linearity).
E. Glossary & Further Reading (short, practical; deeper sources deferred to Book 2)

Part 0 — Orientation & Toolkit

Ch.0 How to Use This Book (and the Twin Volume)

Welcome. This book is the **engineering-first** half of a twin set. It teaches a practical systems playbook that maps the eight primitives of **Incubation Trigram (**先天八卦**)** into familiar engineering levers—**gradients, gates, buffers, boundaries, triggers, guidance, memory, focus**—using dashboards, short-cycle experiments, and lightweight simulators. The **second volume** reuses the same figures, labs, and case cards but overlays a deeper layer (Ô-projection, τ-tick, phase alignment). Here, those ideas appear only as tiny gray **Ô-peek** callouts; you can safely ignore them and still get full value.

What You'll Build

1) A living dashboard
A compact, at-a-glance board you'll update every experiment cycle (we use **12 periods** as a default). It has four bands:

- **Flow** (gradient & gating): throughput, conversion, lead time, abandonment.
- **Stability** (buffers & boundaries): backlog, WIP, cash days, oscillation amplitude.
- **Route** (triggers & guidance): activation rate, route coherence, step-drop index.
- **Depth** (memory & focus): retention slope, resurfacing yield, focus ratio.

2) A 12-period experiment habit
Each chapter includes a **12-period lab** you can run in a spreadsheet or notebook. You'll tweak 1–2 levers (e.g., friction↓, buffer↑), log KPIs, and compare **pre/post variance, recovery time**, and effect sizes. Twelve periods are long enough to see dynamics but short enough to act.

3) Reusable "case cards"
One-page, copy-and-adapt templates that turn a concept into ops:

- **Context & constraints** (what must not change).
- **Objective** (one sentence, one number).
- **Levers** (the specific knobs we'll touch).
- **Failure smells** (early warning diagnostics).
- **Stop-loss rule** (when to revert/abort).

- **Data to log** (columns you'll track for the 12 periods).

4) Four canonical simulators
Lightweight, parameterized simulators you can run in a sheet or Python cell to **preview** behavior and sanity-check lab designs:

1. **Two-Tank Flow** (Gradient & Gate): source ↔ demand with an orifice and friction.
2. **Boundary–Exchange Damper** (Buffers & Rules): inventory/cash as dampers; exchange cadence.
3. **Trigger–Guidance Router** (Nudge & Pathing): event triggers feed a routing matrix.
4. **Memory–Focus Scheduler** (Resurface & Filter): items decay and get resurfaced under a focus budget.

Each sim exposes 4–6 knobs and outputs a few KPIs you'll mirror on your dashboard.

The Chapter Template (How to Read Each Chapter)

Every chapter follows the same five blocks so you can learn fast and deploy faster:

1. **Mechanism Diagram**
   A one-screen schematic that shows the **two or three primitives** in play (e.g., two tanks with a gate and friction; a buffer between exchange partners). Treat it like a circuit diagram: boxes (stocks), arrows (flows), chevrons (gates), and sawteeth (friction).
2. **Minimal Equation**
   A compact, **calibration-friendly** formula that captures the behavior you will measure. Examples:

- Flow with fit & friction:
  $Q \approx \alpha \cdot \Delta V \cdot f(\text{fit}) \cdot (1 - \mu)$
- Buffer sizing under variability: target service level → safety stock term.
- Retention with resurfacing: decay with a refresh impulse each N periods.

The point is not elegance; it's **a small equation you can estimate** from the 12-period log.

3. **KPIs**
   Three to five metrics tied **directly** to the mechanism. We specify **definitions**, **units**, and **alert thresholds** so your dashboard tiles are consistent across chapters. Each KPI has a **why-it-matters** and **how-to-improve** note.
4. **Lab (12-Period Experiment)**
   A recipe you can run this week:

- **Design:** which lever(s) to move, amplitude, and cadence.
- **Controls:** what stays constant (traffic mix, price, SLAs).
- **Data schema:** exact columns to log (see below).
- **Checks:** a quick placebo/A-A sanity test and a fatigue guard.
- **Readout:** how to compute effect size, variance bands, and recovery time.

5. **Case Card**
   A short, realistic scenario (launch ops, content distribution, supply/inventory, or incident containment) implemented with the same KPIs and lab steps. Copy it, tweak the numbers, run it.

   Ô-peek (tiny gray note): Each chapter ends with a one-liner hinting how the **twin volume** will reinterpret the same setup (e.g., what changes when observer roles, cadence ticks, or narrative phase alignment are modeled). Ignore or note for later—your choice.

The Minimal Stack (So You Can Actually Run This)

Option A — Spreadsheet (fastest start).

- One sheet per chapter's **Lab Log** (12 rows = periods).
- A "**KPIs**" sheet that tiles sparklines and thresholds for Flow/Stability/Route/Depth.
- A "**Sims**" sheet with a few input cells and formulas for the four canonical simulators.

Option B — Notebook (Python, if you prefer).

- One cell per simulator (20 lines each).
- A helper function to compute KPIs and thresholds.
- CSV in/out to mirror the spreadsheet schema.

Either way, **keep the file names identical** across chapters so you can swap labs and reuse dashboards.

The 12-Period Lab: Data Schema You'll Reuse Everywhere

Columns (copy/paste into any lab):

- `period` (1–12)
- `lever_1`, `lever_2` (the knobs you adjusted; numeric or categorical)
- `throughput` (units/time or conversions/time)
- `lead_time` (avg or median)
- `abandon_rate` (0–1)
- `buffer_level` (units or days)
- `route_coherence` (0–1 index)
- `step_drop` (largest stage drop %)
- `retention_slope` (Δ over baseline window)
- `resurface_yield` (%)
- `focus_ratio` (signal/attention budget)

- `notes` (free text for anomalies)

Computed fields (formulas provided in each chapter):

- `variance_band` (per KPI)
- `recovery_time` (periods to return within band after a perturbation)
- `effect_size` (pre vs post change; chapter specifies which metric and window)

The Four Canonical Simulators (One-Paragraph Intros)

1) Two-Tank Flow (Gradient & Gate)
Two stocks (source, reachable demand) linked by a gate with friction. Inputs: **ΔV**, **fit**, **friction μ**. Output: throughput Q, lead time. Use it to test whether **reducing friction** or **raising fit** is the better first move.

2) Boundary–Exchange Damper (Buffers & Rules)
A buffer sits between two exchanging parties. Inputs: **reorder point**, **review cadence**, **variability index**. Outputs: fill rate, backlog oscillation. Use it to pick a **buffering policy** that shrinks bullwhip without freezing cash.

3) Trigger–Guidance Router (Nudge & Pathing)
Events hit a routing matrix. Inputs: **trigger intensity**, **guidance stiffness**, **fatigue threshold**. Outputs: activation, route coherence, step-drop. Use it to tune **when** to nudge and **how strongly** to steer.

4) Memory–Focus Scheduler (Resurface & Filter)
Items decay and are resurfaced under a limited **focus budget**. Inputs: **decay rate**, **resurface cadence**, **whitelist density**. Outputs: retention slope, resurfacing yield, focus ratio. Use it to balance **depth vs. breadth**.

Each chapter instantiates one of these with default parameters, so your labs and dashboards always have a **preview model** to compare against reality.

Working Rhythm (Suggested)

1. **Pick a chapter** whose mechanism matches your current bottleneck.
2. **Run the simulator** with your baseline parameters (5 minutes).
3. **Design the 12-period lab** (choose 1–2 levers, set amplitudes).
4. **Update the dashboard** every period; watch **variance bands** and **recovery time**.
5. **Decide** at period 12: lock in, iterate, or revert (follow the stop-loss rule).

Ô-peek Legend (What Those Tiny Gray Notes Mean)

You'll see small gray annotations labeled **Ô-peek**. They're **non-blocking hints** about how the twin volume will reinterpret the same mechanism with three additional ideas:

- **Ô (observer projection):** who is "looking" matters; different roles/frames change what counts as qualified, safe, or salient.

- **τ-tick (cadence):** systems have internal beats; aligning interventions to the beat avoids fatigue and interference.
- **Phase (alignment/lock):** subsystems can lock into productive rhythms or drift into destructive interference.

Formatting:

- **Icon:** ◌Ô
- **Placement:** in the margin or at the end of a section.
- **Length:** one line.
- **Action:** none required. Treat it as a breadcrumb to the twin volume.

Examples you might see:

- ◌Ô **peek:** "If role R observes channel C, the qualification threshold effectively shifts—same mechanism, different counts."
- ◌Ô **peek:** "Nudge at τ/2 tends to generate fatigue; at τ it tends to reinforce memory."
- ◌Ô **peek:** "Guidance stiffness too high can break phase-lock and raise oscillation amplitude."

Common Pitfalls (and How We Avoid Them)

- **Too many levers at once.** In this book, labs move **one or two** knobs only.
- **Dashboard sprawl.** We cap each band at **3–5 KPIs** with fixed definitions.
- **No stop-loss.** Every case card includes a clear **revert condition**.
- **Simulator overtrust.** Sims are for **intuition-building**, not proof; always compare with the 12-period log.

If You Read Only This Chapter

- Clone the **dashboard template**, paste the **12-period schema**, and pick the chapter that matches your current constraint.
- Run one lab this week.
- Ignore the **Ô-peek** if you want. When you're ready to go deeper, the twin volume will use the **same diagrams, labs, and case cards**—just with the projected/cadenced/phase view layered on top.

You're set. Turn the page, pick your dyad, and start the first 12 periods.

Ch.1 Eight Primitives Cheat-Sheet (Trigram → Engineering)

A one-page-per-primitive quick reference. For each dyad you get a **minimal icon**, **what it does**, **the levers you control**, **default KPIs**, **failure smells**, and a **tiny lab** you can run in

12 periods. Use this to decide **which chapter to start with**.

乾×坤 (Heaven–Earth) — Potential Gradient & Capacity Gating

Icon: `[Source]—▷(Gate)—→[Reachable Demand] ; ΔV ↑ → Q ↑`

What it does
Turns **potential difference (ΔV)** into **flow (Q)** through a **gate** under **friction (μ)** and **fit** constraints.

Your levers

- Gate area/throughput coefficient **α**
- **ΔV** (raise useful potential; improve **fit** to reduce mismatch)
- Friction **μ** (policy, UX, legal, latency)
- Quality threshold (what "counts" as qualified)

Minimal equation
$Q \approx \alpha \cdot \Delta V \cdot f(\text{fit}) \cdot (1-\mu)$; with Little's Law: WIP $\approx Q \cdot L$

Default KPIs

- **Throughput (Q)**
- **Lead time (L)**
- **Abandon rate (AR)**
- **Gate utilization (U_g)**
- **Qualified rate (QR)**

Failure smells

- Gate pegged at **U_g ≈ 1** but Q hardly moves (friction bottleneck)
- "**Flood → famine**" oscillation after marketing pushes
- Long-tail **lead time** despite low WIP (hidden batching/hand-offs)
- High **AR** near the gate (fit gap)

12-period micro-lab
P1–6: cut μ by 10% (one friction removal).
P7–12: raise α by 10% (gate widening).
Compare **ΔQ**, **ΔL**, **ΔAR** and pick the higher ROI lever.

Log emphasis → `throughput, lead_time, abandon_rate, lever_1=friction, lever_2=α`

艮×兌 (Mountain–Marsh) — Boundary/Buffer × Exchange

Icon: `[Party A] ⇄ [≡ Buffer] ⇄ [Party B] ; cadence ⏱`

What it does
Uses **buffers** and **exchange rules/cadence** to damp variability, protect cash, and keep service levels.

Your levers

- Buffer size / **safety factor (k)**
- **Reorder point (r)** & review cadence
- Acceptance specs / **boundary rules**
- Exchange batching vs. flow

Minimal relations
Safety stock $\approx z \cdot \sigma_L$; Reorder point $r = d_L + \mathrm{SS}$

Default KPIs

- **Fill rate (FR)**
- **Backlog/stockouts**
- **Oscillation amplitude (OA)**
- **Inventory turns**
- **Cash conversion days (CCC)**

Failure smells

- High inventory **and** high stockouts (spec drift / boundary ping-pong)
- Cash frozen in buffer; **CCC** balloons
- Bullwhip oscillations after small demand shifts
- Endless “exceptions” at the boundary (rule ambiguity)

12-period micro-lab
Hold demand; P1–6: raise **k** (safety) 10%; P7–12: shorten **review cadence**.
Pick the combo minimizing **OA** and **CCC** while keeping **FR ≥ target**.

Log emphasis → `buffer_level, fill_rate, backlog, cash_days, lever_1=k, lever_2=cadence`

> ◌Ô **peek (**山澤通氣 **/ phase interchange):** small phrasing or policy shifts at the **boundary** change what is “safe/legit,” re-phasing exchange so the **same buffer** yields different flows.

震×巽 (Thunder–Wind) — Trigger × Guidance (Routing)

Icon: ⚡ `trigger → ⇝ router → path A/B/C (stiffness γ)`

What it does
Uses **events** to activate users/agents and **guides** them along a route with a tunable **stiffness (γ)** and **throttling** to avoid fatigue.

Your levers

- Trigger intensity / eligibility
- Targeting precision (who gets nudged)
- Guidance **stiffness γ** (how strongly we steer)
- **Cooldown/Throttle** thresholds

Minimal relations
$P_{\text{act}} = \sigma(\beta_0 + \beta_1\text{nudge} - \beta_2\text{fatigue})$; Path entropy H _path falls as γ rises (until overshoot)

Default KPIs

- **Activation rate (A)**
- **Route coherence (R_c)**
- **Max step-drop (D_{max})**
- **Fatigue index (F)**
- **Time-to-value (TTV)**

Failure smells

- High A, poor conversion (**spray-and-pray**)
- Stiff guidance → **resistance** (R_c drops, D_{max} spikes)
- Fatigue waves after campaigns
- Path **thrash** (users bounce between steps)

12-period micro-lab
P1–6: vary **γ** (soft→medium).
P7–12: add **cooldown** rule.
Target ↑**R_c**, ↓**D_{max}**, ↓**F**, ↓**TTV** with minimal A loss.

Log emphasis → `activation, route_coherence, step_drop, lever_1=γ, lever_2=cooldown`

坎×離 (Water–Fire) — Memory × Focus (Attention Control)

Icon: `↻ memory decay + ↻ resurface under ◐ focus budget`

What it does
Manages **decay** and **resurfacing** of items under a limited **focus budget**, balancing breadth vs. depth to sustain **retention** and **recall**.

Your levers

- **Decay rate (δ)** (how fast items fade)
- **Resurface cadence (R)** & dose
- Whitelist density / blacklist rules
- **Focus budget (B_f)** allocation

Minimal relations
$M_{t+1} = (1-\delta)M_t + \text{impulse}(R)$ under $\sum \text{selected} \leq B_f$

Default KPIs

- **Retention slope (m_r)**
- **Resurface yield (Y_r)**
- **Recall latency (t_{rec})**
- **Focus ratio (FR = signal / budget)**
- **Depth-per-user (DPU)**

Failure smells

- **Over-resurfacing** (fatigue, falling Y_r)
- **Over-focus** (stale depth, no new learning)
- **Under-focus** (broad but shallow, DPU stalls)
- Rising **t_{rec}** despite more resurfacing (interference)

12-period micro-lab
Cross two 6-period blocks: (δ fixed)
Block A: **R**↑ at constant **B_f**. Block B: **B_f**↑ at constant **R**.
Pick the policy with ↑**m_r,** ↑**Y_r,** ↓**t_{rec}** and stable **FR**.

Log emphasis → `retention_slope, resurface_yield, focus_ratio, lever_1=R, lever_2=B_f`

Icons & Units (keep it consistent)

- **ΔV** (potential/fit): unitless index or normalized 0–1
- **μ** (friction): 0–1
- **α** (gate coefficient): 0–1 (or capacity/time)
- **k** (safety factor): z-score multiplier
- **γ** (guidance stiffness): 0–1
- **δ** (decay): 0–1 per period
- **B_f** (focus budget): items/period

Legend:

- Gate ▷ Buffer ≡ Boundary | | Trigger ⚡ Router ⇝ Memory ⟳ Resurface ↻ Focus ◐

Picking Your Starting Primitive

- Bottleneck at **entry**? Start with 乾坤.
- Cash/service instability? 艮兌.
- People not following the intended path? 震巽.
- Depth/retention weak or noisy attention? 坎離.

Run the 12-period micro-lab for that dyad first, wire the KPIs into your dashboard, and iterate.

Part I — The Four Dyads

Ch.2 乾×坤 — Gradient & Gate: Two-Tank Flow Control

1) Mechanism (what's happening)

Two reservoirs connected by an orifice:

```
[Source capacity] ──▷(Gate α, friction μ)──→ [Reachable demand]
       ΔV (potential / fit gap)                Q (throughput)
```

- **Source capacity**: your stable ability to supply (production hours, server capacity, reps, bandwidth).
- **Reachable demand**: the part of the market you can actually serve under today's constraints (geo, SLA, price, eligibility).
- **Gate (α)**: any gating/throttling surface (eligibility rule, queue limit, rate limiter, credit screen).
- **Friction (μ)**: UX steps, compliance, latency, handoffs, legal/contract frictions (0–1).
- **Potential difference (ΔV)**: how much the reachable side "pulls" from the source (product–market fit, urgency-to-solve, willingness-to-pay).

Intuition: Flow rises with **ΔV** and **α**, and falls with **μ** and poor **fit**.

---

2) Minimal equation (calibrate, don't worship)

$$Q \approx \alpha \cdot \Delta V \cdot f(\text{fit}) \cdot (1 - \mu)$$

with Little's Law linking flow and delay:

$$\text{WIP} \approx Q \cdot L \quad \Rightarrow \quad L \approx \frac{\text{WIP}}{Q}$$

- $\alpha \in [0, 1]$ (or a capacity/time constant).
- $\mu \in [0, 1]$ friction index (higher = stickier).
- $f(\text{fit}) \in [0, 1]$ captures match quality (targeting, price, promise–delivery match).
- ΔV can be a normalized "pull" index (0–1) or a potential-like score (e.g., qualified demand ÷ supply).

Calibrate $\alpha, \mu, f(\text{fit})$ **from your logs** (see Lab).

3) KPIs (dashboard tiles)

- **Throughput (Q)** — units/time (orders/day, conversions/hr).
- **Lead time (L)** — avg/median delay from "ready" to "done".
- **Abandonment (AR)** — % who enter but fail to pass the gate.
- **Cash days (CCD)** — cash conversion cycle component affected by gating/lead time.

Alert thresholds (defaults, tune per domain)

- $\Delta Q / Q_{\text{baseline}} < +5\%$ after a lever change → **weak effect**
- L not back within **±1.5σ** of baseline in 3 periods → **slow recovery**
- $AR > 30\%$ and rising → **fit or friction problem**
- $\Delta CCD > +5$ days → **starvation or over-gating**

---

4) Instrumentation checklist

- Time-stamped **enter/exit** events at the gate.
- Tag reasons for exit (pass/fail, abandon, timeout).
- Measure **pre-gate latency** (to isolate upstream frictions).
- Separate **eligible** vs **ineligible** demand; log the rule that decided it.
- Track **queue depth (WIP)** at sampling intervals (for L via Little's Law).

---

5) Lab — 12-period experiment (friction↓ vs fit↑)

Goal. Decide which lever yields better ROI: **reduce friction μ** or **raise fit**.

Design. Hold traffic and price steady. Change one lever at a time. Keep staging identical across periods.

Period plan (12 equal periods: days or weeks):

- **P1–2 (baseline):** no changes.
- **P3–8 (Arm A: friction cut):** remove one friction source (e.g., drop a form field, auto-approve trusted cohort). Target $\Delta \mu \approx -10\%$.
- **P9–12 (Arm B: fit raise):** improve targeting/eligibility messaging or qualification logic (e.g., clearer promise, segment-matched landing). Target $\Delta f(\text{fit}) \approx +10\%$.

Data schema (copy these columns):

```
period, lever, friction_mu, fit_f, gate_alpha, potential_dV, throughput_Q, lead_time_L, abandon_rate_AR, cash_days_CCD, WIP, notes
```

Readout (compute each period):

- **Effect size**:
  $\Delta Q = Q_t - \overline{Q}_{\text{baseline}}$,
  $\Delta L = L_t - \overline{L}_{\text{baseline}}$,
  $\Delta AR, \Delta CCD$ analogous.
- **Recovery time**: first period after the lever change when **L** returns within baseline **±1.5σ** and stays there **3 consecutive periods**.
- **Variance band**: rolling σ for Q and L; flag if σ grows >25% (instability).

Decision rule (end of P12):

- If **Arm A** yields $\Delta Q \geq$ Arm B **and** faster recovery **and** $\Delta CCD \leq 0$: prioritize **friction cuts**.
- If **Arm B** yields similar ΔQ but **AR** falls and **L** stabilizes faster: prioritize **fit improvements**.
- If both weak: consider **α** (gate size) or **ΔV** (expand reachable demand) next.

Stop-loss (any period):

- $AR \uparrow 10\%$ **and** $Q \downarrow 10\%$ vs baseline for 2 periods → revert.
- L outside **±3σ** for 2 periods → revert.

---

6) Canonical simulator (use for planning, not proof)

Discrete-time two-tank sketch (per period t):

$$Q_t = \alpha_t \cdot \Delta V_t \cdot f(\text{fit}_t) \cdot (1 - \mu_t)$$
$$\text{WIP}_{t+1} = \max\{0,\ \text{WIP}_t + \text{arrivals}_t - Q_t\}$$
$$L_t \approx \frac{\text{WIP}_t}{\max(Q_t,\ \varepsilon)}$$

- **Inputs:** $\alpha_t, \mu_t, f(\text{fit}_t), \Delta V_t, \text{arrivals}_t$.
- **Outputs:** Q_t, L_t, WIP_t.
- Sensitivities to explore: $\partial Q / \partial \mu$, $\partial Q / \partial f$, and their impact on L.

Excel hints:

- `Q = alpha * dV * fit * (1 - mu)`
- `WIP_next = MAX(0, WIP + arrivals - Q)`
- `L = WIP / MAX(Q, 1e-6)`
  Plot Q and L; overlay ±1.5σ bands.

---

7) Case Card — Launch ops with a staged allowlist gate

Context. You're turning on a new service. You have Source capacity for 1,000 units/week, but real-world reachable demand is uncertain. You'll allowlist cohorts in waves.

Objective. Achieve **Q ≥ 900/wk** with **L ≤ 2 days** and **AR ≤ 20%** by week 4, without increasing **CCD**.

Constraints. Legal requires identity check; support hours fixed; price fixed.

Levers.

- **α (gate size):** allowlist size per wave (25% → 50% → 75%).
- **μ (friction):** optional KYC questions; parallelize checks.
- **f(fit):** messaging per cohort; eligibility clarity.
- **ΔV:** early-access perk to raise pull in reachable segments.

Plan (weeks = periods):

- **W1 (baseline dry-run):** 25% cohort, full KYC, conservative messaging.
- **W2–3 (friction cut):** drop noncritical form fields; batch KYC; expected $\mu \downarrow 10\%$.
- **W4–5 (fit raise):** targeted landing per cohort; eligibility banner; expected $f \uparrow 10\%$.
- **W6 (α raise):** expand allowlist to 50% if $L \leq 2$ days and $AR \leq 20\%$.
- Keep **CCD ≤ baseline**; if it rises, slow allowlist growth.

Failure smells & fixes.

- **Queue pegs (WIP high), L explodes:** α too high for current μ → **pause α**, cut μ, add parallel lanes.
- **AR high in a specific cohort:** fit/messaging mismatch → **tune f**, adjust eligibility text.
- **CCD creeping up:** cash trapped in WIP → **throttle α**, tighten SLA to push Q or re-sequence payments.

Data to log. Cohort id, α, μ components (which frictions removed), fit proxy (match score), ΔV proxy (click/intent index), Q, L, AR, CCD, notes.

---

8) Failure smells (generic) & quick remedies

- **Gate utilization at ~100% but Q flat:** *hidden friction* → instrument pre-gate latency, remove one step.
- **Flood → famine oscillations after pushes:** *over-gating then starvation* → raise α gradually; apply rolling caps.
- **Long lead time at low WIP:** *batching/hand-offs* → unbatch small jobs, create fast lane.
- **High AR clustered at one step:** *fit or comprehension* → rewrite prompt/offer, add live example, relax constraint temporarily.

---

9) What to do next (after this chapter)

- If α and μ changes yield **diminishing returns**, jump to 艮×兌 (buffers & boundaries) to

dampen variability at the boundary.

- If people enter but **don't follow the intended route**, go to 震×巽 (trigger & guidance).
- If you need **deeper engagement over time**, proceed to 坎×離 (memory & focus).

◌Ô **peek (one-liner):** Changing the **observer frame** (who counts and how they count) can **reallocate collapse odds** across channels $P_j(\tau)$ without touching α or μ —same physics here, different effective qualifications there.

Part I — The Four Dyads

Ch.3 艮×兌 — Boundary/Buffer × Exchange: Bullwhip Taming

1) Mechanism (what's happening)

Two parties exchange across a **boundary** with a **buffer** acting as a **damper**:

```
[ Party A ]  ⇄  [  ≡ Buffer  ]  ⇄  [ Party B ]
                |<- service level target ->|
                cadence ⏱, rules | specs | SLAs
```

- **Boundary = rules + specs + cadence.** Defines *what* can pass, *when*, and *in what condition* (acceptance criteria, batching vs. flow, review interval).
- **Buffer = inventory/cash/work-in-process** that absorbs variability so service stays stable.
- **Bullwhip = oscillations** in orders/backlog caused by variability + lags + overreaction at the boundary.

Intuition: Right-sized buffers and crisp rules **damp** noise; sloppy rules and laggy cadence **amplify** it.

---

2) Minimal relations (calibrate, don't worship)

- **Lead-time demand mean & sigma**
  $d_L = \overline{d} \cdot L, \quad \sigma_L = \sqrt{L} \cdot \sigma_d$ (rough, assumes independent periods)
- **Safety stock (service-level z)**
  $\mathrm{SS} = z \cdot \sigma_L$
- **Reorder point (r)**
  $r = d_L + \mathrm{SS}$
- **Order-up-to (S) variant**
  $S = d_{L+R} + z \cdot \sigma_{L+R}$ (if you review every R periods)

  Use **simulation** to estimate **fill rate (FR)** and oscillation; the closed-forms break when demand/lead-time aren't Normal.

---

3) KPIs (dashboard tiles)

- **Fill rate (FR)** — % of demand served on time.
- **WIP / On-hand / Backlog** — levels & trend.
- **Oscillation amplitude (OA)** — peak-to-trough of orders/backlog vs baseline.
- **Inventory turns** — throughput ÷ average inventory.
- **Cash conversion days (CCC)** — DSO + **DIO** − DPO (watch **DIO** when buffers grow).

Alert thresholds (defaults, tune per domain)

- **FR < target − 2%** for 2 periods → undersized or mis-timed buffer.
- **OA ↑ > 25%** period-on-period → boundary/cadence problem.
- **CCC ↑ > +5 days** with no FR gain → cash trapped in buffer.
- **Backlog > 1.5× SS** for 2 periods → reorder policy or spec drift.

---

4) Instrumentation checklist

- Demand per period; lead-time samples; acceptance/reject reasons at the boundary.
- Review cadence stamps (when you evaluate & order).
- Inventory position (on-hand + on-order − backorders).
- Per-period orders placed and fulfilled (to compute OA & bullwhip).
- Cash aging (to compute **DIO** within CCC).

---

5) Lab — 12-period reorder-point sweep

Goal. Choose r (and optionally S) that meets **FR target** with minimal **total cost** and low **OA**.

Controls. Keep demand mix, price, and SLA constant. No emergency expedites unless stop-loss triggers.

Cost model (simple, configurable):
$C_t = h \cdot \overline{\text{inventory}}_t + b \cdot \text{backorders}_t + \ell \cdot \text{lost_sales}_t$

- h: holding cost per unit-period
- b: backorder penalty per unit
- ℓ: lost sale penalty per unit (set $\ell = 0$ if you backorder instead of lose)

Period plan (12 equal periods):

- **P1–2 (baseline):** current r_0, log KPIs, cost, OA.
- **P3–6 (sweep low):** $r = r_0 - 10\%, -20\%$ (two steps, two periods each).

- **P7–10 (sweep high):** $r = r_0 + 10\%, +20\%$ (two steps, two periods each).
- **P11–12 (best candidate):** run the best r^{*} from P3–10 to confirm stability.

Data schema (copy these columns):
`period, r, S, review_cadence, demand, lead_time, on_hand, on_order, backlog, orders_placed, fulfilled, fill_rate, inventory_avg, OA, turns, CCC, cost_holding, cost_backorder, cost_lost, cost_total, notes`

Readout:

- **Surface view:** plot C_{total} vs r; overlay **FR** and **OA**.
- **Decision rule (end of P12):**
  Pick the **lowest cost** r^{*} with **FR ≥ target**, **OA ≤ baseline**, and **CCC ≤ baseline + 3 days**.
- **Stop-loss (any period):** FR < target − 5% **or** OA > baseline + 50% → revert to previous r and shorten review cadence by 1 step.

---

6) Canonical simulator (boundary–exchange damper)

Inventory position (IP) policy:

$$\text{IP}_t = \text{on_hand}_t + \text{on_order}_t - \text{backlog}_t$$
$$\text{if IP}_t < r\text{: place order } q_t = r - \text{IP}_t$$
$$\text{Receive after lead time } L\text{: on_hand}_{t+L} \mathrel{+}= q_t$$

Demand d_t consumes on-hand; if insufficient, either **backorder** or **lose** the shortfall.
Track **FR**, **OA** (var or peak-to-trough of orders/backlog), and **DIO** for **CCC**.

> Tip: Use an **EWMA** of σ to drive adaptive r_t (see "breathing buffers" below).

---

7) Case Card — "Breathing buffers" for spiky content / SKU sets

Context. A media platform pushes irregular **spikes** (events, releases). Inventory is *attention slots* and *service staff hours*; boundary is **publish window + quality spec**. Demand is spiky; lead time to staff up is 1–2 periods.

Objective. Maintain **FR ≥ 95%** and **OA ≤ baseline + 15%** while keeping **CCC** flat (no cash bloat in staff hours or prepaid assets).

Levers.

- **Safety factor** k (via z): base k_0, plus an **adaptive term** for volatility spikes.
- **Review cadence:** weekly → twice weekly during event windows.
- **Acceptance rules:** stricter spec on low-margin items during spikes.

Breathing policy (adaptive SS):
Let $\hat{\sigma}_t = \text{EWMA}(\sigma_t; \lambda)$.

$$\text{SS}_t = z \cdot \sigma_L + \lambda_b \cdot \max(0,\ \hat{\sigma}_t - \sigma_{\text{target}})$$

$$r_t = d_L + \mathrm{SS}_t$$

- When volatility **spikes**, buffer **expands**; when it **settles**, buffer **shrinks** back to cash-efficient levels.
- Pair with **shorter review cadence** only during spike windows to avoid overshoot.

Plan (illustrative 6 weeks = 12 periods):

- **P1–2:** Baseline k_0, weekly review.
- **P3–4 (event spike):** Enable breathing buffers (λ_b > 0), review twice weekly. Tighten acceptance rules for low-margin items.
- **P5–6 (cooldown):** Gradually lower λ_b back to 0; return to weekly review.

Failure smells & fixes.

- **High inventory and frequent stockouts:** spec drift at boundary → clarify acceptance, add "reject reason" taxonomy.
- **OA jumps after spike ends:** cadence still high → revert cadence; reduce SS via λ_b↓.
- **CCC creeps up without FR gains:** buffer fat → shrink *r* 10% and raise turns target temporarily.
- **Exception storms ("manual approvals")**: ambiguous rules → add an auto-decision lane; escalate only edge cases.

Data to log. σ estimates, λ_b value, review cadence flag, accept/reject causes, OA, FR, CCC, margin mix.

---

8) Failure smells (generic) & quick remedies

- **Boundary ping-pong** (A says reject; B says resend): *ambiguous specs* → publish examples + test suite; add "reason codes."
- **Emergency expedites every other period:** *cadence mismatch* → shorten review window or raise SS temporarily via breathing rule.
- **FR target met but OA huge:** *overreaction at r/S* → lower z or add order caps; damp with partial orders.
- **High inventory & high stockouts simultaneously:** *misplaced boundary* → split buffer: fast lane for high-velocity items, quarantine slow movers.

---

9) What to do next (after this chapter)

- If variability is **damped** but people still don't follow the intended path, go to **Ch.4** 震×巽 (triggers & guidance).
- If you're constrained by gate and friction upstream, revisit **Ch.2** 乾×坤 to re-tune α, μ, and **ΔV**.
- If long-term **depth/recall** suffers (knowledge work, learning, loyalty), jump to **Ch.5** 坎×離 (memory & focus).

◌Ô **peek (山澤通氣 / phase interchange):** Tiny changes in boundary wording or review **cadence** re-phase the exchange—same buffer, different **phase alignment**—and the bullwhip shrinks without adding stock.

Ch.4 震×巽 — Trigger × Guidance: Nudge, Route, Convert

1) Mechanism (what's happening)

Event **triggers** activate users/agents; **guidance** steers them along a route. You control **when** to nudge, **who** to nudge, and **how strongly** to steer.

```
⚡ trigger ──> ⇝ router (stiffness γ, throttle ⛔) ──> path A / B / C
↑ eligibility   ↑ hint/tooltips/auto-path                (step 1→2→3→…)
```

- **Trigger:** micro-intervention (email, in-app ping, tooltip, badge).
- **Router:** guidance layer (recommendation, default focus, auto-scroll, prefilled forms).
- **Stiffness γ (0–1):** soft suggestion → hard forcing; high γ can backfire (resistance/fatigue).
- **Throttle/Cooldown:** prevents over-nudging waves.

Intuition: You reduce **activation energy** to start motion, then **reduce path entropy** so people continue along the intended route with fewer stalls.

---

2) Minimal relations (calibrate, don't worship)

Activation (Arrhenius/logit hybrid)

$$P_{\text{act}} = \sigma\left(\beta_0 + \beta_1 \cdot \text{nudge} - \beta_2 \cdot F\right) \quad \text{where} \quad \text{nudge} \sim \Delta E_a$$

Think: nudges **lower** effective activation energy E_a; **fatigue** F raises it back.

Path entropy & coherence

Let p_i be observed branch probabilities at a step;

$$H_{\text{path}} = -\sum_i p_i \log p_i, \qquad R_c = 1 - \frac{H_{\text{path}}}{H_{\max}}$$

Higher R_c = more coherent routing.

Step-drop

$$D_k = 1 - \frac{\text{reach step } k+1}{\text{reach step } k}, \qquad D_{\max} = \max_k D_k$$

---

3) KPIs (dashboard tiles)

- **Activation (A)** — % who take the first intended action post-trigger.

- **Route coherence (R_c)** — 0–1 (normalized entropy).
- **Max step-drop (D_max)** — worst bottleneck between steps.
- **Fatigue index (F)** — proxy: A's slope vs cumulative nudges in last N periods (or response half-life).
- **Time-to-value (TTV)** — median time to the first meaningful success.

Alerts (defaults, tune):

- $\Delta A <$ + 5% after nudge change → weak trigger.
- $R_c \downarrow$ when γ↑ → over-guidance causing resistance.
- $D_{\max} > 40\%$ at any step → redesign that step.
- $F \uparrow$ (A falls with more nudges) → enable cooldown.

---

4) Instrumentation checklist

- **Event table:** trigger_id, audience, send_ts, seen_ts, click/open, acted?.
- **Path tracer:** step_k timestamps; branch chosen; reason if abandon.
- **Guidance registry:** γ level, components (tooltip, default, autofill).
- **Fatigue counters:** nudges_last_7, last_seen_gap, suppression_flag.
- **Denominators:** eligible population per trigger.

---

5) Lab — 12-period experiment (staggered nudges × guidance knobs)

Goal. Improve **A, R_c** and reduce **D_max** while avoiding fatigue.

Controls. Keep traffic mix, pricing, SLAs constant; no overlapping campaigns except the tested nudges.

Period plan (12 equal periods):

- **P1–2 (baseline):** current cadence & γ; record A, R_c, D_max, F, TTV.
- **P3–6 (cadence sweep):** stagger nudges at three send-times (e.g., +0h/+4h/+20h after key event). Add **cooldown** (no more than 1 nudge/24h per user).
- **P7–10 (guidance sweep):** increase **γ** from soft→medium; introduce one structural guide (auto-focus on next field, prefilled template) at step with highest D_k.
- **P11–12 (fatigue guard):** keep best cadence; A/B **γ**=medium vs soft+extra example. Pick the variant with **higher R_c** and **lower F** at equal or higher A.

Data schema:

```
period, nudge_cadence, cooldown_flag, gamma, audience_size, sent, seen, clicked, acted, A, Rc, Dmax, fatigue_index, TTV, notes
```

Readout:

- **Effect size:** ΔA, ΔR_c, ΔD_max vs baseline; **F slope** vs nudges_last_7.
- **Recovery time:** periods to return A, R_c within baseline ±1.5σ after a change.
- **Decision rule (end P12):** choose the **highest A** plan with $R_c \uparrow$, $D_{\max} \downarrow \geq 10\%$, and **no rise in F**.
- **Stop-loss:** if $A \downarrow 10\%$ **and** $F \uparrow$ for 2 periods, revert cadence and lower γ.

---

6) Canonical simulator (trigger–guidance router)

A simple Markov routing sketch with fatigue accumulation:

$$
\begin{aligned}
P_{\text{act},t} &= \sigma(\beta_0 + \beta_1 \cdot \text{nudge}_t - \beta_2 F_t) \\
F_{t+1} &= \rho F_t + \kappa \cdot \text{nudge}_t \quad (0 < \rho < 1) \\
\mathbf{P}_t &= \text{softmax}\big(\mathbf{u} + \gamma_t \, \mathbf{g}\big) \quad \text{(branch probs)} \\
H_{\text{path},t} &= -\sum_i P_{t,i} \log P_{t,i}, \quad R_{c,t} = 1 - \frac{H_{\text{path},t}}{H_{\max}}
\end{aligned}
$$

- **Inputs:** `nudge_t`, `γ_t`, fatigue decay `ρ`, fatigue gain `κ`, guidance vector `g`.
- **Outputs:** `A_t`, `R_c,t`, `D_max,t` from simulated steps.

Use it to preview whether cadence or γ is likely to help **before** you run the 12 periods.

---

7) Case Card — First-session routing for a B2B funnel

Context. New B2B trials must complete: **(1) Invite teammate → (2) Connect data → (3) Create first dashboard → (4) Share**. Current drop at step (2) is 55%.

Objective. Raise **A** (first action within session) from 28%→38%, increase **R_c** by +0.10, and reduce **D_max** at step (2) from 55%→35% in 4 weeks, **without** raising **F**.

Levers.

- **Cadence:** three stagger options on first session start (+0m/+7m/+24h).
- **Targeting:** nudge only admins or identified evaluators.
- **Guidance γ:** soft (tooltip) → medium (auto-focus + inline example) at step (2).
- **Cooldown:** max 1 nudge/24h/user for first 7 days.

Plan (4 weeks = 12 periods):

- **P1–2:** baseline.
- **P3–6 (cadence test):** try +7m nudge (contextual tooltip at step (2)) with cooldown on; log A, R_c, D_max, F.
- **P7–9 (guidance bump):** add prefilled sample connection (γ to medium) + auto-advance to validation.
- **P10–12 (refine):** if F rises, keep γ medium but replace 2nd nudge with “worked

example” video (same cadence).

Failure smells & fixes.

- **A up, R_c flat, D_max unchanged:** nudges spark, but route unclear → increase **specificity** of hint at step (2), not global reminders.
- **R_c up, A down:** over-targeted; widen eligibility or move nudge earlier (+0m).
- **F up:** enable stricter cooldown; replace 3rd nudge with passive guidance (inline checklist).
- **TTV long:** bundle steps (2)+(3) with prefilled template.

Data to log. trigger_id, audience role, send/seen/acted timestamps, chosen branch at each step, γ config, cooldown status, fatigue counters.

---

8) Failure smells (generic) & quick remedies

- **Spray-and-pray:** A up, conversion flat → tighten targeting; reduce nudge count; increase step-specific guidance.
- **Stiffness shock:** γ jump causes route thrash → back to soft; add “why this is recommended.”
- **Fatigue waves:** oscillatory A after campaigns → enforce cooldown, vary content, rotate channels.
- **Path thrash:** users bounce back and forth → lock next-step focus for 10–15s; suppress conflicting UI elements.

---

9) What to do next (after this chapter)

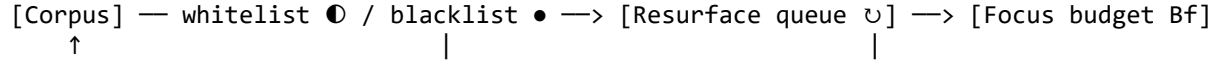
- If people activate and route well but **capacity** or **lead time** choke flow, revisit **Ch.2** 乾×坤.
- If routing is stable but **variability** causes service issues, go to **Ch.3** 艮×兌.
- If early routing works but **long-term depth** is weak, continue to **Ch.5** 坎×離.

◌Ô **peek (one-liner): Phase-lock** between nudge cadence and users’ internal **ticks** improves A and R_c; **desynchrony** inflates fatigue F even when the average nudge count stays the same.

Ch.5 坎×離 — Memory × Focus: Attention as Control Surface

1) Mechanism (what’s happening)

You manage **what gets remembered** (坎) and **what gets foregrounded now** (離). Three controls do the work:



- **Whitelist / Blacklist.** Curate the set eligible to resurface (◐ = eligible; ● = temporarily suppressed).
- **Spaced surfacing.** Schedule **when** an item reappears (fixed | expanding | adaptive).
- **Rehearsal.** Each resurfacing gives a **refresh impulse** that rebuilds memory strength.

Intuition: Memory **decays** unless you **refresh**—but you have a limited **focus budget**. The art is selecting *which* items to resurface *when* so aggregate retention improves without creating fatigue.

---

2) Minimal relations (calibrate, don't worship)

Let item i have memory strength $M_{i,t}$ (unitless, 0–1). Each period:

$$\boxed{M_{i,t+1} = (1 - \delta_i)\, M_{i,t} + \beta\, \mathbf{1}\{i \in \text{resurfaced}_t\}}$$

- **Decay** δ_i: baseline forgetting (0–1 per period).
- **Refresh** β: gain from one effective rehearsal.
- **Focus budget constraint:** $\sum_i \mathbf{1}\{i \in \text{resurfaced}_t\} \le B_f$ (items/period).

Recall probability (for sanity checks, pick one):

$$p_{i,t} = \sigma(\kappa M_{i,t} - \theta) \quad \text{or} \quad p_{i,t} = e^{-\lambda \cdot \text{lag}_{i,t}}$$

Spacing policies (choose one per lab):

- **Fixed:** resurface every s periods.
- **Expanding:** intervals $1, 2, 4, 8, \ldots$ until max window.
- **Adaptive (threshold):** resurface when $\hat{p}_{i,t} < \tau$.

---

3) KPIs (dashboard tiles)

- **Retention curve slope (m_r)** — trend of aggregate recall rate over the 12 periods (↑ is good).
- **Recall latency (t_{rec})** — median time from resurfacing to correct recall/complete action (↓ is good).
- **Focus ratio (FR)** — signal ÷ budget = $\frac{\text{successful recalls}}{B_f}$ (↑ means efficient budget use).

(Optional, often helpful)

- **Resurface yield (Y_r)** — successes ÷ resurfaced items.
- **Interference index (I)** — error or mis-recall rate when closely related items are scheduled together.

Alerts (defaults, tune):

- $m_r \leq 0$ after a spacing change → schedule is wasting budget.
- t_{rec} worsens ≥20% → overstuffed queue or poor timing.
- $FR < 0.5$ for 2 periods → whitelist too dense or items too cold (low M).

---

4) Instrumentation checklist

- **Item registry:** id, value tier (A=core / B=peripheral), topic, last_seen_ts, last_success_ts.
- **Schedule log:** planned interval `s`, actual resurfaced_at, cohort id.
- **Outcome log:** success?, dwell_time, recall_latency, errors, fatigue flags.
- **Budget ledger:** B_f allocated vs used; conflicts (overbooked periods).
- **Whitelist density ρ_w:** $\frac{\text{eligible items}}{\text{total}}$ by tier.

---

5) Lab — 12-period experiment (spacing schedules × whitelist density × bimodal cohorts)

Goal. Improve m_r, reduce t_{rec}, and raise FR by tuning spacing and whitelist density across **bimodal cohorts** (Tier A core items vs Tier B long tail).

Design. 2×2 within 12 periods:

- **Factor 1 — Spacing:** Fixed s vs **Expanding** (1,2,4,…).
- **Factor 2 — Whitelist density ρ_w:** light (20–30%) vs medium (50–60%).
- **Cohorts:** Tier A (core, high-value, lower δ) and Tier B (peripheral, higher δ).

Period plan (12 equal periods):

- **P1–2 (baseline):** current policy; log all KPIs by tier.
- **P3–5 (Block 1):** Fixed s, light ρ_w.
- **P6–8 (Block 2):** Expanding spacing, light ρ_w.
- **P9–10 (Block 3):** Fixed s, medium ρ_w.
- **P11–12 (Block 4):** Expanding spacing, medium ρ_w.

Data schema:

```
period, cohort(A|B), spacing(fixed|exp), s_or_max, rho_w, Bf, resurfaced, successes, m_r,
t_rec, FR, Y_r, interference_I, notes
```

Readout & decision rule (end P12):

- Compute **tier-wise** m_r, t_{rec}, FR.

- Prefer policy with $m_r \uparrow$ **and** $FR \uparrow$ while holding or lowering t_{rec}, especially on **Tier A**.
- If Tier B drags FR below 0.6 in Blocks 3–4, cap ρ_w for Tier B and reserve B_f for Tier A (e.g., 70/30 split).

Stop-loss (any period):

- $FR < 0.4$ **and** $t_{\text{rec}} \uparrow$ 20% vs baseline → revert to previous block.
- **Interference** I **spikes** when similar items are co-scheduled → spread topics (add topic-spread constraint to scheduler).

---

6) Canonical simulator (memory–focus scheduler)

A discrete-time queue with decay and refresh under budget:

$$M_{i,t+1} = (1 - \delta_i)M_{i,t} + \beta \cdot \mathbf{1}\{i \in Q_t\}$$
$$Q_t = \text{top-}B_f \text{ items by priority rule}$$
$$\text{priority}(i) = \begin{cases} \text{time since last seen} & \text{(fixed/exp spacing)} \\ \tau - \hat{p}_{i,t} & \text{(adaptive)} \end{cases}$$

Simulate **Tier A** (δ_A small) and **Tier B** (δ_B larger), and compare m_r, t_{rec}, FR under each block.

---

7) Case Card — "Save or resurface?" content scheduler

Context. A knowledge product shows articles, playbooks, and dashboards. Users can **save** items (pin to top) or let the system **resurface** items later. Budget $B_f = 10$ resurfacing slots per day.

Objective. Raise m_r by +0.08 and FR from 0.55→0.70 over 4 weeks while keeping $t_{\text{rec}} \leq$ baseline.

Levers.

- **Whitelist density** ρ_w**:** start at 30% of corpus eligible; Tier A priority.
- **Spacing: Expanding** for Tier A (1, 2, 4, 8, max 14); **Fixed** $s = 7$ for Tier B.
- **Save vs Resurface rule:**
  - If saved item's **dwell** < 10s twice in a row → unpin and enter resurface queue.
  - If resurfaced item achieves **two consecutive successes** → allow "save" for 3 periods then re-evaluate.

Plan (4 weeks = 12 periods):

- **P1–2:** Baseline mixed schedule; measure KPIs.
- **P3–5:** Apply Tier A expanding spacing; $\rho_w = 30\%$; cap Tier B to 3 of 10 daily slots.
- **P6–8:** If m_r rises but t_{rec} not improving, **lower** s **floor** for items with high value score.

- **P9–12:** Raise ρ_w to 50% **only if** $FR \geq 0.65$; otherwise keep 30% and increase Tier A share to 80%.

Failure smells & fixes.

- **Saved graveyard:** many pins with low dwell → auto-rotate saved items into queue; add "last refreshed" badge.
- **Budget starvation:** Tier B consumes B_f with low Y_r → reduce Tier B slots, or require **topic spread** so Tier A isn't crowded out.
- **Latency creep:** t_{rec} rises → shrink batch size per resurfacing session; interleave short "micro-cards."

Data to log. item_id, tier, saved_flag, last_seen_ts, last_success_ts, dwell, scheduled_interval, outcome(success/fail), latency, topic.

---

8) Failure smells (generic) & quick remedies

- **Over-resurfacing → fatigue:** same users hit too often → introduce **per-user cooldown** and topic diversity.
- **Over-focus → staleness:** Tier A monopolizes budget → reserve 20–30% for exploration.
- **Under-focus → shallow depth:** whitelist too dense → cut ρ_w, increase per-item frequency for top items.
- **Interference between similar items:** co-scheduled topics crowd recall → enforce topic-spread and minimum gap between siblings.

---

9) What to do next (after this chapter)

- If users **start** but don't **advance**, go to **Ch.4** 震×巽 (nudges & guidance).
- If memory/focus works but service wobbles, revisit **Ch.3** 艮×兌 (buffers).
- If inflow and eligibility feel off, return to **Ch.2** 乾×坤 (gradient & gate).

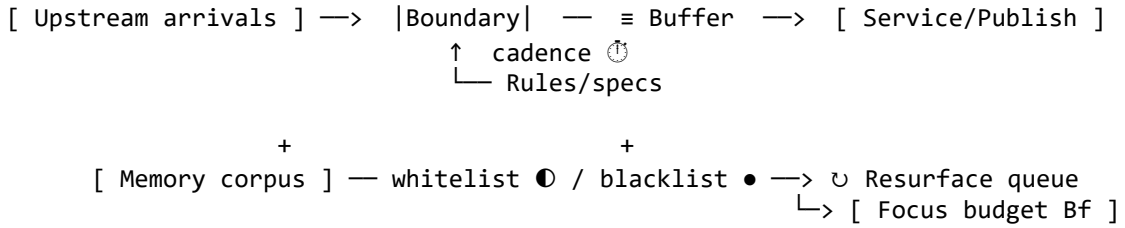
◌Ô **peek (one-liner):** Inside **high-saturation zones** (semantic "BH"), small schedule tweaks behave **near-linearly**—letting you control retention with stable, proportional adjustments even though the full system is nonlinear.

Part II — Two-Dyad × Two-Dyad Modes

Ch.6 Ventilate–Store (艮兌 + 坎離) — Breathing Cycles: Exchange × Memory

1) Mechanism (what's happening)

Combine **boundary/buffer/exchange** (艮兌) with **memory/focus** (坎離) to *breathe* your system:



- **Ventilate (艮兌):** Adjust **exchange cadence** and **buffer size** to damp spikes (bullwhip control).
- **Store (坎離):** Use **spaced resurfacing** and **focus budgeting** to *stage* demand/supply—filling valleys without triggering fatigue.

Intuition: When the boundary threatens to whip (backlog surges), slow the intake and *store* items in memory; when it slackens, **ventilate** by increasing cadence and **resurface** high-value items to keep flow steady.

---

2) Minimal relations (calibrate, don't worship)

Backlog dynamics (at the boundary):

$$\text{backlog}_{t+1} = \max\{0,\ \text{backlog}_t + a_t - s_t\}$$

- $a_t = a_t^{\text{exo}} + r_t$ (exogenous arrivals + **resurfaced items**),
- s_t = service/release at cadence windows (depends on buffer & rules).

Breathing buffer (adaptive safety stock):

$$\text{SS}_t = \text{SS}_0 + \lambda_b \cdot \max(0,\ \hat{\sigma}_t - \sigma^{*}), \quad r_t^{(inv)} = d_L + \text{SS}_t$$

where $\hat{\sigma}_t$ is EWMA of demand variance.

Resurfacing under focus budget B_f:

$$M_{i,t+1} = (1-\delta_i)M_{i,t} + \beta \cdot \mathbf{1}\{i \in Q_t\},\ \sum_{i \in Q_t} 1 \le B_f$$

Backlog half-life (post-shock):

$$t_{1/2} = \min\{\, k:\ \text{backlog}_{t_0+k} \le \tfrac{1}{2}\ \text{backlog}_{t_0} \,\}$$

Oscillation amplitude (OA): peak-to-trough of backlog (or orders placed) over the window.

Control idea:

- If **backlog > target**, **reduce resurfacing** (smaller B_f) and/or lengthen spacing; tighten boundary cadence (slower intake).
- If **backlog < target**, **increase resurfacing** (larger B_f) and/or shorten spacing; open cadence to ventilate.

---

3) KPIs (dashboard tiles)

- **Oscillation amplitude (OA)** — peak-to-trough of backlog/orders vs baseline (↓ is better).
- **Backlog half-life** $t_{1/2}$ — periods to halve backlog after a spike (↓ is better).

(Helpful secondaries)

- **Fill rate (FR)** — should stay ≥ target.
- **Focus ratio (FR_attn)** — successes ÷ resurfaced (don't waste budget).
- **Cash conversion days (CCC/DIO)** — ensure breathing isn't freezing cash.

Alerts (defaults, tune):

- OA > baseline + 25% → boundary/cadence issue.
- $t_{1/2} > 3$ periods for moderate shocks → buffer policy too slow or resurfacing mistimed.
- FR_attn < 0.5 for two periods → whitelist too dense or schedule interference.

---

4) Instrumentation checklist

- **Boundary:** review timestamps, acceptance/reject reasons, order quantities, lead times.
- **Buffer:** on-hand, on-order, backlog; EWMA of demand σ.
- **Memory:** resurfaced list per period, item tiers, dwell/recall outcomes; B_f used.
- **Join keys:** mark which resurfaced items entered the boundary that period (to attribute arrivals r_t).

---

5) Lab — 12-period experiment (buffer cadence × resurfacing rhythm)

Goal. Lower **OA** and $t_{1/2}$ by coordinating boundary cadence with resurfacing rhythm.

Design. 2×2 factorial across 12 periods:

- **Cadence (Boundary): Weekly** vs **Twice-weekly** review/dispatch.
- **Rhythm (Memory): Fixed** resurfacing (every s) vs **Expanding** (1,2,4,… up to max).

Shock. Introduce a controlled spike at **P4** (e.g., +40% exogenous arrivals for one period) to measure $t_{1/2}$.

Period plan:

- **P1–2 (baseline):** Weekly × Fixed.
- **P3–5:** Weekly × Expanding (spike lands at P4).
- **P6–8:** Twice-weekly × Fixed.
- **P9–12:** Twice-weekly × Expanding.

Data schema:

```
period, cadence(weekly|2x), rhythm(fixed|exp), s_or_max, Bf, arrivals_exo, resurfaced, service,
backlog, OA, t_half_marker, FR, FR_attn, CCC, notes
```

Readout & decisions:

- Compute **OA** over each block; measure $t_{1/2}$ after the P4 spike within its block.
- **Pick the policy** with lowest **OA** and $t_{1/2}$ that **maintains FR ≥ target** and **doesn't depress FR_attn**.
- **Stop-loss:** If OA blows out > +50% vs baseline or FR < target − 5%, revert cadence and cut B_f 30% for one period.

Heuristics that usually win: Twice-weekly cadence + Expanding resurfacing, with a **valley-fill rule**: briefly raise B_f when backlog < target band.

---

6) Canonical simulator (coupled damper + resurfacer)

$$a_t = a_t^{\text{exo}} + \min(B_f,\ \text{eligible}_t)$$
$$\text{backlog}_{t+1} = \max\{0,\ \text{backlog}_t + a_t - s_t\}$$
$$s_t = \begin{cases} S & \text{if } t \in \text{review times (per cadence)} \\ s\,\text{min} & \text{otherwise} \end{cases}$$

Resurfacing eligibility follows the memory rule (fixed vs expanding intervals). Vary **cadence** and B_f**/spacing** to see their combined effect on **OA** and $t_{1/2}$.

---

7) Failure smells (and quick fixes)

- **Low OA but long $t_{1/2}$:** cadence too slow to clear spikes → move to twice-weekly *only during* high-σ windows.
- **OA high despite fast cadence:** resurfacing adds load at peaks → add **valley-fill** control (suppress resurfacing when backlog above band; release when below).
- **FR_attn poor:** whitelist too dense or items too cold → shrink ρ_w, increase Tier A share.
- **CCC rising:** buffer bloated → lower SS baseline, keep breathing term (λ_b) modest.

---

8) What to do next (after this chapter)

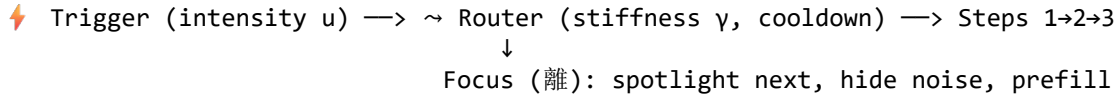
- If backlog oscillation is solved but **people stall at steps**, go to **Ch.4** 震×巽 (nudges & guidance).
- If ventilation works but **inflow/eligibility** still choke, revisit **Ch.2** 乾×坤.
- If long-term **depth/recall** still weak, refine **Ch.5** 坎×離 policies (tiering & adaptive spacing).

◌Ô **peek (one-liner): Phase interchange** at the boundary and **tick pacing** in resurfacing matter—align cadence to the audience's internal **τ** and you'll cut OA and $t_{1/2}$ without adding stock or budget.

Ch.7 Ignite–Guide (震巽 + 離) — Campaign ignition + path steering without churn

1) Mechanism (what's happening)

You **ignite** action with triggers (震) and **guide** the path with steerable focus (巽 + 離). The trick is to get a **clean peak** that **settles into a healthy plateau** while users follow the intended route—no whiplash, no fatigue.



- **Ignite:** campaign bursts, in-product nudges, channel fan-out.
- **Guide:** defaults, inline examples, auto-focus, gentle hiding of off-path options.
- **Focus (離):** what is foregrounded now (one-next-step), not everything at once.

Intuition: Peak is your **match strike**; plateau is the **steady flame**. Over-steer (γ too high) or over-nudge (u too high) makes smoke (fatigue/churn).

---

2) Minimal relations (calibrate, don't worship)

Activation with fatigue feedback

$$A_t = \sigma\left(\beta_0 + \beta_1 u_t - \beta_2 F_t\right), \quad F_{t+1} = \rho F_t + \kappa u_t \ \ (0 < \rho < 1)$$

Routing coherence via guidance

$$\mathbf{P}_t = \mathrm{softmax}(\mathbf{u}_0 + \gamma\,\mathbf{g}), \quad R_c = 1 - \frac{H(\mathbf{P}_t)}{H_{\max}}$$

Peak/Plateau ratio (PPR)

$$\mathrm{PPR} = \frac{\max_t \in \mathrm{burst} A_t}{\overline{A}_\mathrm{plateau}}$$

Desirable band: **1.2–1.8** (visible spark, sustainable burn).

---

3) KPIs (dashboard tiles)

- **Peak/Plateau Ratio (PPR)** — peak after ignition ÷ average plateau (target 1.2–1.8).
- **Route coherence (R_c)** — 0–1; higher means users follow the intended path.

Helpful secondaries

- **Max step-drop (D_max)** — worst drop between successive steps.
- **Fatigue index (F)** — response decay vs recent nudges.
- **Churn risk (CR)** — unsubs/complaints/opt-outs per 1k nudges.

Alerts (defaults, tune):

- PPR < 1.1 → ignition too weak; PPR > 2.2 → flash-in-pan (expect crash).
- $R_c \downarrow$ when γ↑ → over-steer; expect higher D_max.
- $CR > 3/1000$ or F↑ across 2 periods → enforce cooldown.

---

4) Instrumentation checklist

- Trigger ledger: `trigger_id, u (intensity), audience, channel, send_ts.`
- Guidance registry: `γ level, components (prefill, auto-focus, hide-elsewhere).`
- Path tracer: timestamps per step, chosen branch, drop reason.
- Fatigue/churn: per-user nudge count (7d), unsub/complaint, open-but-ignore streak.
- Denominators: eligible population & suppression rules.

---

5) Lab — 12-period experiment (trigger intensity × guidance stiffness)

Goal. Find a **u × γ** combo with **PPR in band** and **higher** R_c, while **not** elevating fatigue/churn.

Design. 2×2 factorial across 12 periods:

- **u (intensity):** Low vs High (e.g., 1 vs 3 touches per user per window; or 1× vs 2× channel fan-out).
- **γ (stiffness):** Soft (hints/examples) vs Medium (prefill + auto-focus; no hard locking).

Period plan:

- **P1–2 (baseline)**: current u, γ; measure PPR, R_c, D_max, F, CR.
- **P3–5:** u=Low × γ=Soft (clean spark test).
- **P6–8:** u=Low × γ=Medium (steer more, same fuel).
- **P9–12:** u=High × γ=Soft (more fuel, gentle steering).
  (If resources allow, run u=High × γ=Medium in a parallel A/B cohort; otherwise hold for next cycle.)

Guardrails (always on): cooldown=1 nudge/24h/user; no more than 2 channels within 6h; suppression for recent non-responders.

Data schema:
`period, u, gamma, eligible, sent, seen, acted, A, PPR, Rc, Dmax, F, CR, TTV, notes`

Readout & decision rule (end P12):

- Prefer the cell with **PPR 1.2–1.8**, $R_c \uparrow \geq +0.05$, $D_{\max} \downarrow \geq 10\%$, **no rise in CR/F**.
- If both Low-u cells hit goals, pick **γ=Medium** only if R_c benefit ≥ +0.03; else keep Soft (saves guidance build).

Stop-loss (any period): PPR > 2.4 **and** $R_c \downarrow$ → reduce u by 50% next period; if CR > 5/1000, halt burst.

---

6) Canonical simulator (burst + guidance + fatigue)

$$A_t = \sigma(\beta_0 + \beta_1 u_t - \beta_2 F_t), \quad F_{t+1} = \rho F_t + \kappa u_t$$
$$\mathbf{P}_t = \text{softmax}(\mathbf{u}_0 + \gamma\,\mathbf{g}),\ R_c = 1 - \frac{H(\mathbf{P}_t)}{H\,\max}$$
$$\text{PPR} = \frac{\max_t \in \text{burst} A_t}{\overline{A}_\text{plateau}}$$

Simulate a 3-period burst (t=3–5) then hold cadence; observe PPR, R_c, F under each (u,γ) pair.

---

7) Failure smells (and quick fixes)

- **Tall peak, collapsing plateau (PPR > 2.2):** too much u → cut channels, add cooldown, move value earlier (reduce TTV).
- **R_c drops when γ rises:** over-steer → switch from hard hints to **worked examples**; keep agency.
- **D_max stuck at a step:** step-specific friction; fix the step (prefill, inline validator) rather than global u.
- **CR climbs:** opt-outs → channel mismatch; switch to in-product hints, reduce audience breadth.

---

8) What to do next (after this chapter)

- If plateaus are uneven due to supply/backlog swings, pair with **Ch.6 Ventilate–Store**.
- If inflow eligibility/gating constrains peak entirely, revisit **Ch.2** 乾×坤.
- If long-term depth still lags, refine **Ch.5** 坎×離 (tiering & adaptive spacing).

◌Ô **peek (one-liner):** Campaigns have **phase-lock windows**—nudge during the audience's τ-aligned moments and medium γ yields high R_c with PPR in band; off-phase bursts inflate fatigue for the same u.

Ch.8 Seal–Bleed (乾坤 + 艮兌) — Gate hard where it matters; bleed where it pays

1) Mechanism (what's happening)

Blend **Gradient & Gate** (乾坤) with **Boundary/Buffer** (艮兌):

```
[ Inflow (candidates) ] -- score q̂ --> ▷ Hard Gate θ_g --> [ Main lane ]
                                  \-->  ↘ Bleed valve (cap b) --> [ Bleed lane ]
                                      (near-band, lower SLA/spec, buffered release)
```

- **Seal (Hard Gate θ_g):** admit only high-quality/fit traffic to the **main lane**; protect

brand/SLA.

- **Bleed (Valve cap b):** route **near-miss** traffic into a **buffered, lower-SLA** lane to harvest value, smooth spikes, or learn.
- **Boundary rules/cadence:** define specs for both lanes and when bleed is allowed.

Intuition: Seal the core to keep **precision** high; open a controlled **bleed** to capture upside and reduce bullwhip—*but only when the economics are net positive*.

---

2) Minimal relations (calibrate, don't worship)

Lane assignment

$$\text{Main if } \hat{q} \geq \theta_g; \quad \text{Bleed if } \theta_b \leq \hat{q} < \theta_g \text{ and } b_t > 0$$

with θ_b a near-band floor (optional) and b_t the bleed capacity this period.

Expected economics

$$\text{Precision}_{\text{main}} = \frac{\text{TP}}{\text{TP+FP}}, \quad \text{Recall}_{\text{main}} = \frac{\text{TP}}{\text{TP+FN}}$$

$$Y_{\text{leak}} = \frac{\sum_{\text{bleed}} (\pi_{\text{rev}} - c_{\text{serve}} - \text{penalty}_{\text{SLA}} - \text{brand_risk})}{\#\text{bleed}}$$

- TP/FP/FN labeled by **observed outcome** (pass/fail, return, complaint, churn).
- Y_{leak} = **leakage yield** per unit; must be **> 0** after penalties.

Backpressure-aware bleed (optional)

$$b_t = \min\left(b_{\max},\ k \cdot \max(0,\ \text{backlog}_t - \text{target})\right)$$

Open the valve only when backlog high; shut when low.

---

3) KPIs (dashboard tiles)

- **Quality Gate Precision (main)** — TP / (TP+FP).
- **Quality Gate Recall (main)** — TP / (TP+FN).
- **Leakage Yield** Y_{leak} — net profit per bleed unit.

Helpful secondaries

- **Leakage rate** — % of inflow sent to bleed.
- **Return/complaint rate (main & bleed)** — brand/SLA health.
- **CCC / DIO** — watch cash tied in bleed buffers.
- **OA (orders/backlog)** — bullwhip after gating changes.

Guardrails (defaults, tune):

- Precision$_{\text{main}} \geq$ **0.92**; Recall$_{\text{main}} \geq$ **0.65** (example B2B SaaS).
- $Y_{\text{leak}} > 0$ for 2 consecutive periods before widening bleed.
- Complaint rate$_{\text{bleed}} \leq$ **2×** main's rate.

---

4) Instrumentation checklist

- **Scoring:** $\hat{q}$ per candidate; lane taken; threshold θ_g used.
- **Outcomes:** pass/fail, revenue, serve cost, SLA credits, returns, complaints.
- **Buffers:** bleed backlog level, release cadence, service time.
- **Confusion matrix logging:** for main lane; **label near-band holdouts** via periodic audit samples to estimate FN.
- **Financials:** Y_{leak} components; CCC/DIO by lane.

---

5) Lab — 12-period experiment (gate threshold × bleed valve size)

Goal. Find a (θ_g, b) pair that preserves **main precision**, acceptable **recall**, and **positive** Y_{leak} —while avoiding bullwhip.

Design. 2×2 factorial across 12 periods:

- **Gate threshold θ_g: High** (seal harder) vs **Low** (wider main).
- **Bleed cap b: Small** (5–10% inflow) vs **Medium** (15–25%).

Period plan (3 periods per cell):

- **P1–3:** θ_g=High, b=Small.
- **P4–6:** θ_g=High, b=Medium.
- **P7–9:** θ_g=Low, b=Small.
- **P10–12:** θ_g=Low, b=Medium.

Controls. Keep pricing and marketing mix constant; same review cadence; bleed lane has explicit SLA/spec and signage.

Data schema:

```
period, theta_g, bleed_cap_b, inflow, main_qty, bleed_qty, TP, FP, FN_est, precision_main,
recall_main, Y_leak, complaints_main, complaints_bleed, backlog_bleed, OA, CCC, notes
```

Readout & decision rule (end P12):

- **Eliminate** any cell with Precision$_{\text{main}}$ < target **or** $Y_{\text{leak}} \leq 0$.
- From survivors, pick the cell with **highest net profit** and **stable OA/CCC**.
- If two tie, prefer **higher recall** (future growth) and **lower complaint rate**.

Stop-loss (any period):

- Complaint rate$_{\text{bleed}}$ > **2×** main **or** $Y_{\text{leak}} < 0$ for 2 periods → halve b next period.
- Precision$_{\text{main}}$ drops by ≥2 pts → raise θ_g one notch immediately.

---

6) Canonical simulator (seal–bleed with outcomes)

For each candidate with score $\hat{q}$:

$$\text{lane} = \begin{cases} \text{main}, & \hat{q} \geq \theta_g \\ \text{bleed}, & \theta_b \leq \hat{q} < \theta_g \text{ and } b_t > 0 \\ \text{reject}, & \text{otherwise} \end{cases}$$

$$\Pr(\text{good} \mid \hat{q}, \text{lane}) = \sigma(a + c\hat{q} - d \cdot \mathbf{1}\{\text{lane} = \text{bleed}\})$$

$$\pi_{\text{unit}} = \begin{cases} \pi_{\text{rev}} - c_{\text{serve}} - \text{penalty}_{\text{SLA}} - \text{brand_risk}, & \text{good in bleed} \\ \pi_{\text{rev}} - c_{\text{serve}} - \text{return/complaint cost}, & \text{bad} \end{cases}$$

Aggregate to compute **precision/recall**, Y_{leak}, and **OA** under each (θ_g, b).

---

7) Failure smells (and quick fixes)

- **High inventory & high stockouts (both lanes):** boundary ambiguity → tighten specs, add reason codes; separate SKUs/cohorts.
- **Great precision, poor recall, flat revenue:** gate too tight → lower θ_g one notch **or** add bleed with **strict cap**.
- **Positive Y_{leak} but complaints spike:** mis-sold bleed lane → clearer SLA, separate branding, or downgrade promise.
- **OA spikes after widening bleed:** open valve only on **backpressure** (use b_t rule), or increase bleed release cadence.

---

8) What to do next (after this chapter)

- If peaks/plateaus from campaigns misbehave, pair with **Ch.7 Ignite–Guide** to shape bursts without churn.
- If backlog oscillation drives bleed misuse, revisit **Ch.6 Ventilate–Store**.
- If hard-gated inflow still starves the system, re-tune **Ch.2** 乾×坤 (friction/fit/ΔV).

◌Ô **peek (one-liner):** Change the **observer frame** (who counts, how counted) and the same traffic yields different *"qualified"* sets—your apparent precision/recall shift without touching θ_g or b.

Ch.9 Pulse–Soak (震巽 + 坎) — Short pulses; long soak into memory

1) Mechanism (what's happening)

You **ignite short pulses** (震巽：triggers & guidance) and then **soak** those impressions into memory (坎：rehearsal under a focus budget).

```
Pulse (width w, intensity u)  →  route & act  →  Tail (carryover)
                                        |
                                Soak (resurface cadence S; budget Bf)
```

- **Pulse:** brief, concentrated outreach (burst emails, in-app banners, PR spike).
- **Route:** guidance that reduces path entropy during the burst.
- **Soak:** planned resurfacing after the pulse so impressions **consolidate** into recall/retention—without exhausting attention.

Intuition: The pulse buys **attention now**; the soak converts it into **memory later**. Too long a pulse cannibalizes the soak; too short with no soak wastes the win.

---

2) Minimal relations (calibrate, don't worship)

Immediate response with fatigue

$$A_t = \sigma(\beta_0 + \beta_1 u_t - \beta_2 F_t), \qquad F_{t+1} = \rho F_t + \kappa u_t \quad (0 < \rho < 1)$$

Carryover (tail) from the pulse — impulse response h_k (e.g., geometric decay):

$$\text{inc_rev}_t = \sum_{k=0}^{K} h_k \, u_{t-k}, \quad h_k = \eta \lambda^k, \; 0 < \lambda < 1$$

Memory with soak (per item/cohort i, aggregated in practice):

$$M_{i,t+1} = (1-\delta) M_{i,t} + \beta \cdot \mathbf{1}\{i \in \text{resurfaced}_t\}, \qquad \sum_{i \in \text{resurfaced}_t} \leq B_f$$

KPIs we'll compute

- **Pulse ROAS** (during burst + attributed tail):

$$\text{ROAS}_{\text{pulse}} = \frac{\sum_{t \in \mathcal{W}_{\text{pulse}}} (\text{rev}_t - \text{baseline}_t) + \sum_{k=1}^{K} g_k \, (\text{rev}_{t_0+k} - \text{baseline}_{t_0+k})}{\text{spend}_{\text{pulse}}}$$

with g_k an attribution weight (often $g_k = h_k / \sum h_k$).

- **Soak–Retention Δ** (change after the soak window S):

$$\Delta R_{\text{soak}} = R_{\text{post-}S} - R_{\text{baseline}}$$

Helpful secondaries: **Carryover ratio** CR_{tail} = attributed tail/in-burst; **Lift half-life** $t_{1/2}$ of incremental response.

---

3) KPIs (dashboard tiles)

- **Pulse ROAS** — efficiency of the burst including its short tail.
- **Soak–Retention Δ** — how much retention improved after resurfacing.

Watch as guardrails: **Fatigue index** F, **complaints/opt-outs**, **path step-drop** during burst.

Alerts (defaults, tune):

- $\text{ROAS}_{\text{pulse}} < 1.2 \rightarrow$ weak burst or poor targeting.
- $\Delta R_{\text{soak}} \leq 0 \rightarrow$ soak timing wrong or whitelist too dense.
- $F \uparrow$ and opt-outs > 3/1000 during burst → reduce width or add cooldown.

---

4) Instrumentation checklist

- **Pulse ledger:** start/end, spend, channels, audience, u (intensity), w (width).
- **Holdout cohorts** (A/B) or pre/post baselines to estimate incremental rev.
- **Path tracer** during burst: steps, branch, step-drop.
- **Soak schedule:** resurfacing timestamps, cadence S, focus budget B_f.
- **Retention panels:** cohort assignment; retention at fixed anchors (e.g., D+7, D+14).

---

5) Lab — 12-period experiment (pulse width × soak window)

Goal. Find a pair (w, S) that yields $\textbf{ROAS}_{\text{pulse}}\uparrow$ and $\Delta R_{\text{soak}}\uparrow$ with low fatigue.

Design. 2×2 factorial via **cohorts**, so you get enough post-pulse time to measure soak.

- **Pulse width w: Short** (1 period) vs **Long** (3 periods).
- **Soak window S: Short** (single resurfacing at +1 period) vs **Long** (resurfacing at +1 and +3).

Timeline (12 equal periods):

- **P1–2 (baseline):** measure rev & retention; no campaigns.
- **P3–5 (pulses):**
  - **Cohort A:** Short (w=1 at P3).
  - **Cohort B:** Long (w=3 at P3–5).
    Identical targeting & channels; cooldown = 1 nudge/24h/user.
- **P6–12 (soak & readout):** split each cohort into **S-short** (resurface at P7) and **S-long** (resurface at P7 and P9). Track retention at P10 and P12.

Data schema:

```
period, cohort(A|B), subcohort(Ss|Sl), w, S, u, spend, exposures, rev, baseline_rev, inc_rev, ROAS_pulse, fatigue_F, complaints_per_1k, resurface_flag, retention_anchor(P10|P12), ΔR_soak, notes
```

Readout & decision rule (end P12):

- Choose (w, S) with **highest ROAS**$_{\text{pulse}}$ among cells where $\Delta R_{\text{soak}} > 0$ and **fatigue/ complaints don't rise**.
- If tie on ROAS, pick larger ΔR_{soak}.
- **Stop-loss:** if $F \uparrow$ and complaints > 3/1000 in any active cell → cut u by 50% and end pulse early; skip next resurface for that cell.

Heuristic you'll often see: **Short pulse + long soak** wins when value requires learning; **Long pulse + short soak** can work for time-sensitive promos but risks fatigue.

---

6) Canonical simulator (burst → tail → soak)

$$A_t = \sigma(\beta_0 + \beta_1 u_t - \beta_2 F_t), \quad F_{t+1} = \rho F_t + \kappa u_t$$

$$\text{inc_rev}_t = \sum_{k=0}^{K} h_k \, u_{t-k}, \quad h_k = \eta \lambda^k$$

$$M_{t+1} = (1-\delta) M_t + \beta \cdot \mathbf{1}\{t \in \text{resurface times}\}$$

$$\Delta R_{\text{soak}} \propto M_{t_{\text{anchor}}} - M_{t_{\text{baseline}}}$$

Feed **u_t** with the pulse shape (w) and schedule resurface times per **S**; compute ROAS and ΔR_{soak}, track F.

---

7) Failure smells (and quick fixes)

- **Great ROAS, zero soak gain:** spacing or audience wrong → lengthen S, narrow whitelist, add worked examples to resurface.
- **Soak lifts, ROAS poor:** pulse too weak or ill-timed → raise u slightly or align to higher-intent moments; keep S.
- **High fatigue/opt-outs:** reduce width (w=1), enforce cooldown, switch channels to in-product hints.
- **Cannibalization of organic:** baseline dips during pulse → add holdout; restrict pulse to incremental audiences.

---

8) What to do next (after this chapter)

- If bursts cause **routing stalls**, pair with **Ch.4** 震×巽 to fix step-level friction.
- If post-burst load overwhelms service, combine with **Ch.6 Ventilate–Store** to dampen OA and shorten backlog half-life.
- If qualification is the bottleneck, revisit **Ch.2** 乾×坤 (gate & fit).

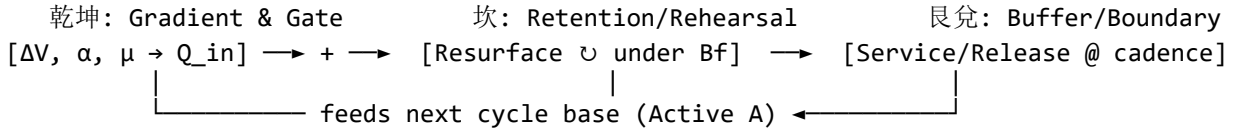
◌Ô **peek (one-liner): Latent iT** (imaginary-time) buildup before internal **ticks** makes certain moments "soak-ready"—pulses just before those ticks convert to memory with outsized efficiency.

Part III — Triads (the “compounding kits”)

Ch.10 Compounding Trio: Gradient + Retention + Buffer

1) Mechanism (what’s happening)

You connect three levers so growth **compounds** instead of sputters:



- **Gradient (乾坤):** raise **ΔV** (pull/fit), widen **α** (gate), cut **μ** (frictions) → more **qualified inflow** Q.
- **Retention (坎):** resurface with smart spacing under **focus budget** B_f → a larger fraction **returns** next cycle.
- **Buffer (艮兌):** right-size **safety stock & cadence** so service stays smooth—no bullwhip, no cash freeze.

Compounding intuition: When **retention lift** and **inflow** both land **inside a stable service window**, the active base grows **multiplicatively** cycle over cycle.

---

2) Minimal relations (calibrate, don’t worship)

Let A_t be active/engaged base at period t.
Qualified inflow $Q_t \approx \alpha \cdot \Delta V \cdot f(\text{fit}) \cdot (1-\mu)$.
Deliverable service S_t comes from capacity + buffer policy (capped by cadence).

$$\textbf{Delivered}_t = \min\{Q_t + R_t^{\text{soak}},\ S_t\}$$
$$A_{t+1} = R_t \cdot A_t \ + \ \textbf{Delivered}_t$$

- R_t — effective **retention multiplier** from your resurfacing policy.
- R_t^{soak} — resurfaced/returning items that behave like inflow.
- S_t — **release/service** permitted by boundary & buffer this period.

Net Compounding Factor (NCF) — normalized per-cycle growth:

$$\boxed{\text{NCF}_t \ \equiv \ \frac{A_{t+1}}{A_t} \ = \ R_t + \frac{\min\{Q_t + R_t^{\text{soak}},\ S_t\}}{A_t}}$$

If $\text{NCF}_t > 1$ **and** stability constraints hold (below), you’re compounding.

Stability region (safe-operating conditions)

- **Utilization:** $\rho_t = \frac{\textbf{Delivered}_t}{S_t} \le 0.85$ (avoid queue blowups).

- **Variance band:** rolling σ of Q, A, backlog ≤ **+25%** of baseline.
- **Backlog half-life:** $t_{1/2} \leq 2$ after a moderate demand shock.
- **Fatigue & complaints:** within guardrails of Ch.4/Ch.9.
- **CCC/DIO:** cash days not degrading (> +3d) under the policy.

Hysteresis risks (why escalation ≠ easy reversal)

- **Over-buffers** leave cash trapped; backing down doesn't free it instantly.
- **Over-wide gates** flood low-fit users; churn memory (R_t) lags to recover.
- **Over-resurfacing** induces fatigue; even if you stop, response rebounds slowly.

---

3) KPIs (dashboard tiles)

- **Net Compounding Factor (NCF)** — target **> 1.05** sustained (example).
- **Variance bands** — rolling σ for Q, A, and backlog vs baseline (aim ≤ +25%).
- **MTTR** (mean time to recovery) — periods to return within **±1.5σ** after a lever change or shock (aim ≤ 2).

Helpful secondaries: utilization ρ, backlog half-life $t_{1/2}$, CCC/DIO, fatigue index.

---

4) Instrumentation checklist

- **Gradient:** ΔV proxy, α, μ, fit score; Q_t.
- **Retention:** resurfaced count, spacing mode, B_f, per-tier outcomes → compute R_t.
- **Buffer:** safety factor k, reorder point r, review cadence; S_t, backlog, OA, $t_{1/2}$.
- **Active base:** A_t panel at fixed anchors (weekly/monthly).
- **Events:** explicit "shock" markers when you change a knob.

---

5) Lab — 12-period 3-knob sweep (find the safe-operating envelope)

Goal. Identify combinations of **Gradient gain (G)**, **Retention lift (R)**, and **Buffer strength (B)** that keep you **inside stability** while maximizing **NCF**.

Knobs (two levels each):

- **G (Gradient): Low** vs **High** via ΔV / α (keep μ steady).
- **R (Retention): Low** vs **High** via spacing policy & B_f (Tier A priority).
- **B (Buffer): Low** vs **High** via safety factor k & review cadence.

Design: 2×2×2 factorial with foldover (8 cells) + baseline + confirmations → 12 periods.

Period plan:

- **P1–2 (baseline):** current G,R,B; measure σ bands & MTTR on a tiny probe.
- **P3–6 (set A):**
  - P3: G–, R–, B–
  - P4: G+, R–, B+
  - P5: G–, R+, B+
  - P6: G+, R+, B–
- **P7–10 (foldover set B):**
  - P7: G+, R–, B–
  - P8: G–, R+, B–
  - P9: G–, R–, B+
  - P10: G+, R+, B+
- **P11–12 (confirm top 2):** run the two best cells again to verify **NCF** and **stability**.

Controls & guards:

- Hold price, mix, and SLAs constant.
- Cooldown rules from Ch.4/Ch.9 always on.
- No emergency expedites unless stop-loss triggers.

Data schema:

```
period, G_level, R_level, B_level, Q, S, Delivered, A, NCF, rho, var_Q, var_A, backlog, t_half, MTTR, CCC, fatigue, notes
```

Readout & decision rules:

1. **Stability filter:** keep cells with $\rho \leq 0.85$, variance bands ≤ +25%, $t_{1/2} \leq 2$, and CCC ≤ baseline +3d.
2. **Pick top** by median **NCF** across its runs (P? and P??).
3. **Tie-breakers:** lower MTTR → lower variance → lower CCC.

Stop-loss (any period):

- $\rho > 0.95$ **or** variance band breach **or** CCC jump > +5d → step B to Low immediately; if still unstable, step G to Low next period.

Heuristics that usually win:

- **G: High, R: High, B: Medium** (not max) often yields best **NCF** with manageable variance; a too-strong **B** can suppress flow and bloat CCC.

---

6) Canonical simulator (triad coupling)

A compact coupling of the earlier chapters:

$$
\begin{aligned}
Q_t &= \alpha_t \, \Delta V_t f(\text{fit}_t)(1-\mu) \\
S_t &= S_0 + \phi(k_t,\ \text{cadence}) - \psi(\text{volatility}) \\
\textbf{Delivered}_t &= \min(Q_t + R_t^{\text{soak}},\ S_t) \\
A_{t+1} &= R_t A_t + \textbf{Delivered}_t \\
\text{NCF}_t &= \frac{A_{t+1}}{A_t}
\end{aligned}
$$

Shock the system (e.g., spike in Q_t) and compute **variance bands**, $t_{1/2}$, and **MTTR** as you traverse cells.

---

7) Failure smells (and quick fixes)

- **NCF > 1 but variance bands blown:** G too high for B → reduce G one notch; increase cadence instead of static k.
- **Stable but NCF ≈ 1:** R too low → raise Tier A resurfacing (expanding spacing) before widening G.
- **CCC creeping up:** over-buffered → lower k or lengthen spacing on low-value resurfacing; keep G steady.
- **MTTR > 3:** cadence mismatch → move boundary review to twice-weekly during volatility; add valley-fill rule for resurfacing.

---

8) What to do next (after this chapter)

- If inflow bursts are the main driver, tune bursts with **Ch.7 Ignite–Guide** and soak with **Ch.9 Pulse–Soak**.
- If service/backlog wobble, revisit **Ch.6 Ventilate–Store**.
- If the gate is mis-qualifying, re-fit **Ch.8 Seal–Bleed** (precision/recall vs leakage yield).

◌Ô **peek (one-liner):** When **ΔV (campaign cadence)**, **resurfacing rhythm**, and **boundary review cadence** share a **τ-aligned cycle**, compounding stabilizes —phase alignment spreads load evenly and raises sustained **NCF** without extra spend.

Ch.11 Crisis Trio: Trigger + Boundary + Memory (Firebreaks)

1) Mechanism (what's happening)

You build a **four-step firebreak** that turns incidents into fast recoveries and durable learning:

```
Trigger (detect/anomaly)  →  |Boundary| Isolate (circuit-break, rate-limit)
                               ↘ Reroute (safe path / degrade)
                                 → Repair (rollback/fix)
                                   → Rehearse (postmortem → spaced drill)
```

- **Trigger (震):** detect, page, auto-runbook.
- **Boundary (艮兌): isolate** blast radius (circuit breakers, feature flags, quota walls) and **reroute** traffic to safe lanes.
- **Memory (坎):** codify what worked, then **rehearse** on a schedule (spaced drills) so response gets faster and cheaper.

Flow: isolate → reroute → repair → rehearse. The first three contain the fire; the last one makes the *next* fire smaller.

---

2) Minimal relations (calibrate, don't worship)

Let I_t be incident "intensity" (e.g., error rate × impact), B_t backlog/impact stock, and R_t a response readiness factor improved by rehearsal.

Containment dynamics

$$I_{t+1} = (1 - \phi - \rho_r)\, I_t - r_{\text{fix}}\, I_t$$
$$B_{t+1} = \max\{0,\ B_t + \gamma\, I_t - s_t\}$$

- ϕ = **isolation factor** from boundary (0–1): how much coupling you cut.
- ρ_r = **reroute fraction** sent to safe lanes/degraded mode (0–1).
- r_{fix} = **repair rate** (rollback/patch).
- γ converts intensity to backlog/impact; s_t is service/release during incident.

Readiness improves with rehearsal (spaced memory)

$$R_{t+1} = (1 - \delta)R_t + \beta \cdot \mathbf{1}\{t \in \text{drill times}\}$$

and $r_{\text{fix}} = r_0 + k_R R_t$, trigger latency $T_{\text{page}} = T_0 - h_R R_t$.

KPIs we'll compute

- **Containment time (CT):** first period k with $I_{t_0+k} \le \tau_I$ *and* B_{t_0+k} trending down (within ±1.5σ band).
- **Spill cost (SC):**

$$SC = \sum_t \left(c_u \cdot \text{unserved}_t + c_{\text{SLA}} \cdot \text{breaches}_t + c_b \cdot \text{brand/complaint}_t\right)$$

- **Learning carryover (LC):** improvement that persists to the **next unrelated** incident:

$$LC = \frac{CT_{\text{before}} - CT_{\text{after}}}{CT_{\text{before}}} \quad \text{(and similarly for } SC\text{)}$$

---

3) KPIs (dashboard tiles)

- **Containment time (CT)** — periods to sub-threshold intensity with backlog declining.

- **Spill cost (SC)** — cumulative economic damage (unserved, SLA, brand).
- **Learning carryover (LC)** — normalized reduction in CT (and SC) on the next drill/incident.

Helpful secondaries

- **Blast radius (%)** — affected traffic share before isolation.
- **Trigger latency (T_page)** — detection→action time.
- **Reroute efficiency** — kept-throughput / intended-throughput during incident.

Alerts (defaults, tune):

- CT > 2 periods for moderate incidents → isolation/reroute weak.
- SC rising across drills → repairs not addressing root causes.
- LC ≈ 0 after two rehearsals → drills not retained (spacing or realism off).

---

4) Instrumentation checklist

- **Detector stream:** anomaly type, threshold, page time, ack time, auto-runbook id.
- **Boundary toggles:** breaker/flag states, rate limits, quotas, who flipped and when.
- **Reroute ledger:** fraction diverted, safe lane performance, degradation mode chosen.
- **Repair diary:** rollback/patch timestamps, tests passed, blast radius after repair.
- **Memory logs:** postmortem issues → actions → **drill schedule**; drill outcomes (latency, errors, surprises).
- **Accounting hooks:** unserved units, SLA credits, complaint count → SC.

---

5) Lab — 12-period incident drills with rotating weak links

Goal. Reduce **CT** and **SC** and show **positive LC** by practicing firebreaks across different subsystems (avoid overfitting to one failure).

Design. Each period is either a drill or a normal run. You'll **rotate the weak link** (DB, cache, third-party API, feature flag gone bad) and **randomize the failure mode** (slowdown vs hard fail).

Period plan (12 equal periods):

- **P1–2 (baseline readiness):** measure current CT/SC with a light "probe drill" on one subsystem; record R_t.
- **P3–4 (Drill A — DB slow):** scripted isolation (read-only, breaker on write), reroute to cache layer, rollback if needed; postmortem → actions → schedule rehearsal at P8.
- **P5–6 (Drill B — third-party API fail):** quota wall + stub fallback; degrade non-critical features; postmortem → actions → rehearsal at P9.

- **P7 (Normal + surprise mini-incident):** introduce 10% response slowdown in the *other* path; check whether CT improved *without* explicit rehearsal.
- **P8 (Rehearsal A):** run DB scenario again; expect ↓CT, ↓SC.
- **P9 (Rehearsal B):** run API scenario again; expect ↓CT, ↓SC.
- **P10 (Drill C — feature-flag misconfig):** isolate cohort, safe default, hotfix flow; rehearse at P12.
- **P11 (Normal readout):** compute LC from A/B; sanity-check spill trends.
- **P12 (Rehearsal C):** run feature-flag scenario again; finalize LC.

Controls. Keep traffic mix realistic; disable marketing bursts; no infra changes except planned toggles.

Data schema:

```
period, scenario(DB|API|FLAG|Normal), failure_mode(slow|hard), page_time, ack_time,
trigger_latency, isolation_phi, reroute_frac, repair_rate, intensity_I, backlog_B,
containment_time_CT, unserved, SLA_credits, complaints, spill_cost_SC, readiness_R,
rehearsal_flag, carryover_LC_CT, carryover_LC_SC, notes
```

Readout & decision rule (end P12):

- **Pass** if median **CT ≤ 2** for moderate drills, **SC ↓ ≥ 25%** vs baseline probes, and **LC ≥ 30%** for at least two distinct scenarios.
- Prioritize actions that improved **both** CT and SC (not just one).
- **Stop-loss (any period):** if a drill causes **blast radius > 30%** or **complaints > 5/1k**, abort drill, revert toggles, and review runbook before next period.

---

6) Canonical simulator (firebreak sandbox)

A compact update each period:

$$
\begin{aligned}
I_{t+1} &= (1 - \phi_t - \rho_{r,t} - r_{\text{fix},t})\, I_t + \epsilon_t \\
B_{t+1} &= \max\{0,\ B_t + \gamma I_t - s_t\} \\
R_{t+1} &= (1-\delta) R_t + \beta \cdot \mathbf{1}\{t \in \text{drills}\} \\
r_{\text{fix},t} &= r_0 + k_R R_t, \quad T_{\text{page},t} = T_0 - h_R R_t
\end{aligned}
$$

Drive ϕ_t with breaker/flag actions, $\rho_{r,t}$ with your reroute cap, and let drills raise R_t. Read CT, SC, LC from the traces.

---

7) Failure smells (and quick fixes)

- **Pager storms / flapping:** trigger thresholds too tight → add **debounce** and multi-signal confirmation.
- **Isolation works, reroute collapses:** safe lane under-provisioned → pre-warm cache/cdn, cap reroute to $\rho r_{\max}$ with backpressure.
- **Great CT, high SC:** you saved time but lost money → refine degrade plan (protect high-value flows first), speed refunds to cut brand cost.

- **No LC across drills:** postmortems not turning into drills or spacing too short/long → adopt **expanding rehearsal** (1, 2, 4 periods).
- **Boundary ambiguity at 2AM:** runbooks unclear → convert to **toggle checklists** with exact owners and time limits.

---

8) What to do next (after this chapter)

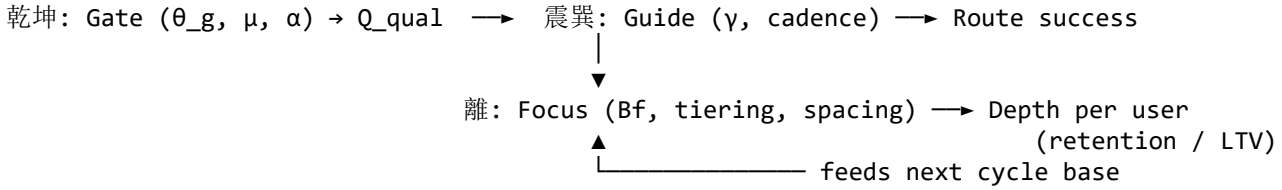
- If post-incident backlog lingers, pair with **Ch.6 Ventilate–Store** (adaptive cadence & breathing buffers).
- If incidents arise during campaigns, coordinate with **Ch.7 Ignite–Guide** or **Ch.9 Pulse–Soak** to avoid synchronized stress.
- If gating lets too many risky requests through, revisit **Ch.8 Seal–Bleed** for stricter main-lane precision.

◌Ô **peek (one-liner):** Watch **collapse entropy spikes** in observables (errors, queuing, complaint text)—they often precede saturation; tripping the firebreak *before* the spike peaks keeps CT and SC in the stable, low-cost regime.

Ch.12 Growth Flywheel: Gate + Guide + Focus — *qualify → steer → deepen*

1) Mechanism (what's happening)

You chain three levers so **qualified flow** becomes **guided success**, then **deepens** into durable value. Run this loop continuously.



- **Gate** (乾坤): tighten fit, reduce friction → more **qualified velocity** (Q_qual/time).
- **Guide** (震巽): raise route coherence with the lightest control that works.
- **Focus** (離): allocate limited attention to deepen the highest-value users/units first.

Intuition: A great gate without guidance **leaks**; guidance without focus **thins**; focus without qualified inflow **stalls**. The flywheel needs all three every cycle.

---

2) Minimal relations (calibrate, don't worship)

Qualified velocity (QV)

$$\mathrm{QV}_t \approx \alpha_t \cdot \Delta V_t \cdot f(\mathrm{fit}_t) \cdot (1-\mu_t) \cdot \mathbf{1}\{\hat{q} \geq \theta_{g,t}\}$$

Route efficiency (R_c) from Ch.4

$$R_{c,t} = 1 - \frac{H(\mathbf{P}_t)}{H\,\max}, \quad \mathbf{P}_t = \mathrm{softmax}(\mathbf{u}_0 + \gamma_t\,\mathbf{g})$$

Depth-per-user (DPU) (simple additive proxy)

$$\mathrm{DPU}_t = w_1 \cdot \mathrm{features_used}_t + w_2 \cdot \mathrm{success_events}_t + w_3 \cdot \mathrm{retention_anchor}_t$$

driven by **focus budget** B_f and tiering/spacing (Ch.5).

Flywheel step (per period):

$$\mathbf{Delivered}_t = \min\{\mathrm{QV}_t \cdot R_{c,t},\ S_t\}, \qquad A_{t+1} = A_t + \mathbf{Delivered}_t \cdot \mathrm{DPU}_t$$

(Use S_t from boundary capacity if relevant; otherwise drop the min.)

---

3) KPIs (dashboard tiles)

- **Qualified velocity (QV)** — qualified entries/time after gate.
- **Route efficiency (R_c)** — coherence of the intended path.
- **Depth-per-user (DPU)** — composite of feature depth / success events / retention anchor.

Helpful guardrails

- **Fatigue index (F)**; **Complaints/1k**; **Precision**$_{\text{main}}$ if gating touches quality.
- **CCC/DIO** when focus/guidance create backlogs.

Alert bands (defaults, tune):

- QV flat after friction/fit change → mis-specified threshold θ_g or wrong channel.
- $R_c \downarrow$ when γ↑ → over-steer; expect step-drop spikes.
- DPU stalls at high R_c → focus budget flowing to low-value tiers.

---

4) Instrumentation checklist

- **Gate:** $\hat{q}$, θ_g, pass/fail reasons, μ components, α changes; QV.
- **Guide:** γ level, components (prefill/auto-focus/examples), nudge cadence, route traces.
- **Focus:** B_f per tier, spacing policy, resurfaced items, successes, retention anchors.
- **Attribution:** link each delivered unit from *gate → guide → focus* so DPU credit is assigned to the path that produced it.

---

5) Lab — Multi-arm bandit for guidance × focus budget (12 periods)

Goal. Learn a combo of **guidance pattern** and **focus allocation** that maximizes the flywheel without breaching guardrails.

Arms (examples, define 6–8 total):

- **Guidance pattern (G):**
  - G1: Soft hints + examples (γ=soft)
  - G2: Prefill + auto-focus (γ=medium)
  - G3: Soft + sequenced checklists (γ=soft, step-specific)
- **Focus allocation (F):**
  - F1: TierA:TierB = **70:30** with expanding spacing for TierA
  - F2: **80:20** (TierA heavier)
  - F3: **60:40** with topic-spread constraint (reduce interference)

Reward (composite, normalized 0–1):

$$\text{Reward}_t = \lambda_1 \widetilde{\text{QV}}_t + \lambda_2 \widetilde{R_{c,t}} + \lambda_3 \widetilde{\text{DPU}}_t$$

Choose λ's (e.g., 0.4/0.3/0.3). Penalize breaches: subtract π_F if fatigue or complaints exceed bands; subtract π_P if $\text{precision}_{\text{main}}$ drops.

Algorithm (recommend): Thompson Sampling (handles noise & delayed DPU).
Fallback: **UCB1** with score $= \bar{r} + c\sqrt{\frac{\ln t}{n}}$.

Period plan (12 equal periods):

- **P1–2 (warmup/explore):** pull each arm once; compute provisional reward.
- **P3–10 (TS/UCB):** let the bandit select arms; enforce guardrails (cooldown, precision).
- **P11–12 (confirm):** lock top-2 arms 50/50 to validate lift and check stability (variance bands, MTTR to baseline after a knob change).

Data schema:

```
period, arm_id, guidance(G1|G2|G3…), focus(F1|F2|F3…), pulls, QV, Rc, DPU, reward, fatigue_F, complaints_per_1k, precision_main, CCC, notes
```

Decision rule (end P12):
Pick the arm with highest **median reward** that **does not** breach guardrails. If two tie, pick higher **DPU** at equal **QV** (depth compounds).

Stop-loss (any period):

- Complaints > 5/1k **or** $\text{precision}_{\text{main}}$ −2 pts → immediately back off γ and/or tighten θ_g.
- Fatigue trend ↑ 2 periods → cut nudge cadence by 50% in the next pull.

---

6) Canonical simulator (flywheel A/B sandbox)

$$\mathrm{QV}_t = \alpha\,\Delta\,Vf(\mathrm{fit})(1-\mu)\cdot \mathbf{1}\{\hat{q} \geq \theta_g\}$$
$$R_{c,t} = 1 - \frac{H(\mathrm{softmax}(\mathbf{u_0} + \gamma\,\mathbf{g}))}{H\,\mathrm{max}}$$
$$\mathrm{DPU}_t = g(B_f,\ \mathrm{tiering},\ \mathrm{spacing})$$
$$\mathrm{Reward}_t = \lambda_1 \widetilde{\mathrm{QV}}_t + \lambda_2 \widetilde{R_{c,t}} + \lambda_3 \widetilde{\mathrm{DPU}}_t - \mathrm{penalties}$$

Run it with your arm definitions to sanity-check which mixes are even plausible before live testing.

---

7) Failure smells (and quick fixes)

- **QV↑, R_c↑, DPU flat:** focus budget wasted on low-value tiers → shift to **F2 (80:20)** and apply expanding spacing to TierA.
- **QV flat across arms:** gate is the true constraint → revisit θ_g, μ, or channel fit (Ch.2/ Ch.8).
- **R_c drops when γ rises:** over-steer → switch to **G1/G3** (worked examples, checklists) and keep agency.
- **DPU↑ but CCC worsens:** depth creating service/backlog → add cadence/valley-fill from Ch.6; cap deep actions per period.

---

8) What to do next (after this chapter)

- Need burst ignition? Pair with **Ch.7 Ignite–Guide** and then re-optimize arms.
- Supply wobble from depth work? Add **Ch.6 Ventilate–Store** to keep OA and backlog half-life in band.
- Quality drift at the gate? Re-tune **Ch.8 Seal–Bleed** to protect precision while preserving recall.

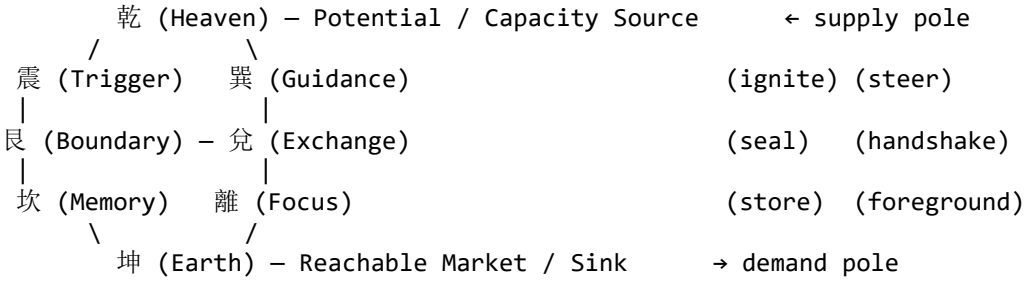
> ◌Ô **peek (one-liner):** As the flywheel stabilizes, you'll see **attractor formation**—operationally visible as increasing **phase curvature** in path choices (coherence rises with smaller nudge/focus changes).

Part IV — Four-in-One: The Eight-Node Operating Diagram

Ch.13 The Eight-Node Control Board (先天八卦 as Ops Map)

1) What this board is

A single-page **ops map** that shows your whole system as **eight nodes** (the 先天八卦 ring), with **flows** between them. It answers three executive questions at a glance:
Who supplies? Who sinks? Where does it block?
You'll instrument each node with probes, allocate a **friction budget**, and define **risk walls** (automated breakers).



Opposites (先天): 乾↔坤, 震↔巽, 坎↔離, 艮↔兌.
Radials carry your **main throughput**; chords carry **control** (nudges, cadence, buffers).

---

2) Roles of the eight nodes (and what to probe)

- 乾 · **Heaven (Source / Capacity / ΔV) — supplier**
  Probe: capacity, α (gate coeff), ΔV (pull/fit), μ_upstream (friction).
  Watch: utilization, warm-start time, cost curve.

- 坤 · **Earth (Market / Sink) — consumer**
  Probe: reachable demand, qualified rate, cash days (CCD).
  Watch: abandonment, segment mix, elasticity.

- 震 · **Thunder (Triggers) — spark**
  Probe: cadence, audience, A (activation), fatigue F.
  Watch: PPR (peak/plateau), opt-outs, spillover to support.

- 巽 · **Wind (Guidance) — steering**
  Probe: γ (stiffness), R_c (route coherence), D_max (step-drop).
  Watch: resistance at high γ, time-to-value.

- 艮 · **Mountain (Boundary) — seal**
  Probe: rule hits/misses, breaker flips, isolation φ.
  Watch: exception storms, ping-pong rejections.

- 兌 · **Marsh (Exchange) — handshake**
  Probe: fill rate, backlog, OA (oscillation amplitude).
  Watch: spec drift, cadence slip, DIO within CCC.

- 坎 · **Water (Memory) — store**
  Probe: decay δ, refresh β, resurfaced/period, m_r (retention slope).
  Watch: interference, cold-item drag.

- 離 · **Fire (Focus) — foreground**
  Probe: B_f (attention budget), Y_r (resurface yield), DPU (depth/user).
  Watch: saturation (crowding), topic spread.

---

3) Flows, blocks, and the friction budget

Link model (each edge j):
Throughput $Q_j \approx \alpha_j \cdot \Delta V_j \cdot f(\text{fit}_j) \cdot (1 - \mu_j)$

- **Capacity** C_j and **utilization** $\rho_j = Q_j / C_j$ tell you where you're tight.

- **Friction budget:** $\sum \mu_j \leq \mu_{\text{total}}$. You choose where friction is **protective** (艮 main gate) vs **wasteful** (pre-gate UX, duplicate checks).

- **Where it blocks:** look for saturated cuts (min-cut intuition). If any edge on the 乾→…→坤 path runs $\rho_j \geq 0.9$ or spikes in **variance**, that's your primary choke.

Default cut checks (weekly):

- **Supply cut:** 乾→震/巽→艮;
- **Control cut:** 震/巽→離;
- **Delivery cut:** 兌→坤 with 坎/離 injections.
  Flag "**red-cut**" if two adjacent edges exceed 0.9 utilization or OA rises >25%.

---

4) Instrumentation: probes per node + edge

- **Node probes** as above (8 mini-tiles on the board).
- **Edge probes** (hover/expand in your dashboard): $Q_j, C_j, \rho_j, \mu_j, \Delta V_j, f(\text{fit}_j)$, variance band, MTTR after a change.
- **Event overlays:** campaigns (震), policy pushes (艮/兌), resurfacing windows (坎/離), capacity changes (乾).
- **Cut visual:** highlight any min-cut estimate and annotate the **top two** edges by load and variance.

---

5) Risk walls (automated breakers)

Define **hard stops** and **soft brakes** on nodes/edges:

- **Hard stops (breakers):**
  - **Precision floor** at 乾→艮: if precision_main < target, raise θ_g now.
  - **Complaint wall** at 兌→坤: if complaints > X/1k, halve bleed cap.
  - **Fatigue wall** at 震/巽: if F↑ two periods, enforce cooldown.
  - **Backlog wall** at 兌: if backlog > band, slow intake (艮) and pause resurfacing (坎).
- **Soft brakes (dampers):**
  - **Breathing buffers** (艮/兌): adapt safety stock via EWMA of σ.
  - **Valley-fill** (坎/離): raise B_f only when backlog below band.
  - **Guidance easing** (巽/離): auto-lower γ if R_c drops.

Each wall/brake gets: **trigger**, **action**, **owner**, **expiry/review**.

---

6) Building your board (one-week, step-by-step)

Day 1 — Draw & name: place the eight nodes; name your **main product flow** 乾→…→坤.
Day 2 — Wire edges: list the 6–10 edges you actually use; add Q, C, μ placeholders.
Day 3 — Drop probes: attach the node KPIs; define alert bands.

Day 4 — Friction budget: enumerate all frictions; mark **protective** vs **wasteful**; set μ_{total} caps by quarter.
Day 5 — Risk walls: implement the four hard stops + two soft brakes; dry-run breaker playbooks.
Day 6 — Cut check: compute current min-cut and label the **top choke**; propose the next lever (friction cut vs buffer vs guidance).
Day 7 — Ops ritual: 30-min review cadence; assign owners for any node that crossed band.

---

7) Using the board in weekly ops

1. **Scan**: red-cut? any node outside band?
2. **Decide**: one **unlock** (remove wasteful μ), one **stabilize** (buffer/cadence), one **deepen** (focus on Tier A).
3. **Arm links**: if you push 震 (campaign), pre-arm 兌's breathing buffer and set 坎/離 valley-fill.
4. **Post-action MTTR**: confirm you're back within ±1.5σ in ≤2 periods—else revert.

---

8) Quick diagnostics (where it blocks)

- **High AR near gate, Q flat:** pre-gate μ wasteful → cut UX steps; keep 艮 protective μ intact.
- **OA at 兌 with healthy supply:** cadence mismatch → shorten review only during high-σ windows.
- **R_c falls as γ rises:** over-steer → swap to worked examples; keep agency.
- **DPU flat at good R_c:** focus misallocated → raise Tier A share, enforce topic spread.
- **CCC creeping up:** over-buffered → lower k, delay low-value resurfacing.

---

9) Minimal math cheats (optional section on the board)

- **Edge health:** $\rho_j = Q_j / C_j$, **OK** if ≤0.85.
- **Path bound:** $Q_{\text{path}} \le \min_j C_j(1-\mu_j)$.
- **Friction budget:** reduce μ where **R_c** or **DPU** aren't harmed; keep μ where **precision** or **risk** demand it.
- **Variance band:** flag if rolling σ > 1.25× baseline on any two adjacent edges.

◌Ô **peek (one-liner):** The eight nodes behave like **attractors** of a semantic OS; as you tune cadence and observation, paths curve toward stable channels—Book 2 overlays this with x, θ, τ and phase geometry (preview only).

Ch.14 Synchronization, Drift, and Debt — *Align cadences; detect drift; pay down oscillatory debt*

1) Mechanism (what's happening)

Your eight-node board has **multiple clocks**: launch/marketing (震) runs in bursts, boundary reviews (艮/兌) run on a fixed cadence, memory resurfacing (坎/離) has its own rhythm, and capacity changes (乾) follow ops calendars. When these **ticks** aren't aligned, you get **drift** (relative phase slip) and **beats** (amplitude modulations) that inflate queues, fatigue, and complaints—creating **oscillatory debt** you must pay down later.

```
震 (burst τz)  → load
艮/兌 (review τb)  → release/clear
坎/離 (resurface τm)  → background load

 misaligned τ's  ⇒  drift  ⇒  beats  ⇒  oscillations  ⇒  debt
```

Goal: choose **harmonic cadences**, keep **phase offsets** small, and use **valley-fill** to bleed oscillations before they accrue as debt.

---

2) Minimal relations (calibrate, don't worship)

Subsystem clocks & phase

- Each subsystem i has a **natural period** T_i and **phase** $\phi_i \in [0, 2\pi)$.
- **Clock skew (pairwise):** $\Delta\tau_{ij} = \min_k |(t_i - t_j) + k \cdot \mathrm{lcm}(T_i, T_j)|$ (time offset between their event series).
- **Phase drift rate:** $\dot{\phi}_{ij} = \frac{d}{dt}(\phi_i - \phi_j)$. If $|\dot{\phi}_{ij}| \to 0$ you are **phase-locked**.

Beat period (two rhythms)

$$T_{\text{beat}} = \left| \frac{1}{T_1} - \frac{1}{T_2} \right|^{-1}$$

Large, slow "breathing" waves at T_{beat} expose alignment issues.

Collapse delay proxy (trigger → KPI response)

- Cross-correlate a trigger series x_t with a KPI y_t. **Collapse delay** D is the lag at which correlation peaks.

Saturation index (simple, actionable)

$$\mathrm{SI} = \frac{\rho}{1-\rho}, \quad \rho = \frac{Q}{C}$$

When **SI > 5** ($\approx \rho > 0.83$), small time-phase slips create large queue oscillations.

Oscillatory debt (costed area above band)

$$\mathrm{OD} = \sum_t w_t \cdot \max\{0,\ |x_t - \bar{x}| - \mathrm{band}\}$$

Pick x = backlog, complaints, or fatigue; w_t = economic weight.

---

3) KPIs (dashboard tiles)

- **Clock skew** (Δτ pairs): 震↔艮/兌, 震↔坎/離, 坎/離↔艮/兌. Target **≤ 10–20%** of the shorter period.
- **Collapse delay proxies (D)**: trigger→Q, trigger→complaints, resurface→retention. Keep **stable** (variance ≤ +25%).
- **Saturation index (SI)** for the delivery cut (兌 path): keep **SI ≤ 5** (≈ ρ ≤ 0.83).

Helpful secondaries

- **Beat amplitude** at T_{beat} (peak/trough of backlog).
- **Variance band** breaches count per month.
- **Debt ledger**: OD for backlog, fatigue, complaint credits.

Alerts (defaults, tune):

- Any Δτ > 25% of min period for 2+ weeks → **drift** likely harmful.
- **SI** > 5 for 2 periods → expect nonlinear queue growth.
- **Beat amplitude** > +25% of baseline → cadence mismatch.

---

4) Instrumentation checklist

- **Event stamps**: campaign pulses, boundary reviews, resurfacing batches, capacity changes.
- **Per-edge flows**: Q, C, ρ, variance; backlog & MTTR.
- **Cross-corr panel**: automatic lag estimation for (trigger→Q), (trigger→complaints), (resurface→retention).
- **Spectral glance** (optional): weekly FFT of backlog/orders to surface $T_1, T_2, T_{\text{beat}}$.
- **Debt ledger**: OD by metric with cost weights.

---

5) Playbook — Synchronization in three moves

Move A — Pick harmonic cadences

- Choose a **master meeting cadence** (e.g., weekly).
- Set **boundary review** to **weekly** or **twice-weekly** (a harmonic).
- Set **resurfacing** to **weekly with expanding intervals** (1–2–4–8) anchored on the master day.
- Restrict **campaign pulses** to the **same weekday/time** unless testing off-phase on purpose.

Move B — Set phase offsets

- Aim for **small, fixed offsets**: ϕ_{pulse} slightly *before* ϕ_{boundary} so capacity opens near demand peaks; schedule **resurface** near valleys to valley-fill.
- Target Δτ(震→艮/兌) ≈ **+0.5–1 day**; Δτ(震→坎/離) ≈ **+1–2 days**.

Move C — Add brakes

- If **SI** rising, **lower pulse intensity** (u) or **delay** by one slot; enforce cooldown.
- If **beat amplitude** large, **tighten boundary cadence** during peaks and **suppress resurfacing** (坎/離) above backlog band.

---

6) Lab — 12-period alignment & debt paydown

Goal. Reduce **clock skew**, stabilize **collapse delay**, lower **SI**, and retire **OD** by tuning offsets and cadence combinations.

Design. 3 phases across 12 equal periods (days/weeks):

- **P1–3 (diagnose):** measure Δτ, D, SI, OD under current cadences.
- **P4–7 (align):**
  - Lock **boundary** to weekly (or 2×/wk) on a fixed day.
  - Shift **campaign pulse** by +0.5–1 day before boundary.
  - Anchor **resurfacing** to valley slots (post-boundary +1–2 days).
- **P8–12 (pay down):**
  - **Debt sprint:** valley-fill resurfacing + moderate service cadence increase; throttle pulses by −30%.
  - Maintain guardrails (fatigue, complaints).

Data schema:

```
period, T_pulse, T_bound, T_mem, phi_pulse, phi_bound, phi_mem, Δτ_pulse-bound, Δτ_pulse-mem,
D_trigger→Q, D_trigger→complaints, SI, beat_amp, backlog, OD_backlog, OD_fatigue,
OD_complaints, actions, notes
```

Readout & decision rule (end P12):

- **Pass** if Δτ pairs ≤ **20%** of min period, **D** variance ≤ **+25%**, **SI ≤ 5**, and total **OD ↓ ≥ 40%** vs baseline.
- If **SI** still high, slow pulses further and/or widen service windows; keep resurfacing in valleys.

Stop-loss (any period):

- Complaints spike > 3/1k or fatigue slope ↑ two periods → pause pulses one slot and halve resurfacing that cycle.

---

7) Failure smells (and quick fixes)

- **Pretty cadences, ugly queues:** you aligned calendars, not **capacity** → increase release quantity or add a second review during peaks.
- **Delay wobble (D swings):** inconsistent routing or measurement → stabilize guidance (γ), fix denominators, re-run cross-corr.
- **Beat remains huge:** two independent pulses (e.g., PR & lifecycle) desync → designate a single **pulse owner** or hard-block one to valley slots.
- **Debt won't fall:** you keep adding load while "paying down" → enforce a **debt sprint**: throttle inflow, hold marketing, boost service for two cycles.

---

8) Weekly ritual (15 minutes)

1. **Skew check:** show Δτ and T_{beat}.
2. **Delay check:** trigger→Q lag stable? complaints lag stable?
3. **SI & OD:** any edge >5 on SI? OD slope negative?
4. **One action each:** shift one phase, tweak one cadence, retire one debt bucket.

> ◌Ô **peek (one-liner):** Teams carry **semantic clocks**; when observers change, the **felt time** of the org changes—phase-locked groups move "fast," drifted groups feel "slow" even at equal raw speed.

Part V — Domain Playbooks

Ch.15 Software Delivery — Feature Gating, Rollout Buffers, Incident Firebreaks

Mechanism

Software delivery is a flow system. Code enters from source (capacity), passes through review and test gates (filters), accumulates in staging buffers (inventory), and exits into production (demand). Like any flow, it risks overload (too many features), starvation (blocked teams), or rupture (incident cascade).

Three primitives map directly:

- 乾坤 **(Gradient & Gate)** → Feature gating, staged rollout.
- 艮兌 **(Boundary & Buffer)** → Rollout buffers, blue/green pools, canaries.
- 震巽 × 坎離 **(Trigger + Memory/Focus)** → Incident triggers, firebreak rehearsals, postmortem learning.

Delivery is healthiest when each gate has calibrated throughput, each buffer breathes (absorbs load without stalling), and each trigger reroutes failures quickly.

Minimal Equations

1. **Throughput Flow**

$$Q \approx \alpha \cdot \Delta V \cdot f(\text{fit}) \cdot (1 - \mu)$$

- ΔV: capacity–demand gradient (features ready vs. slots open)
- $f(\text{fit})$: alignment with gate criteria (test coverage, review pass)
- μ: friction (handoffs, manual steps)

2. **Buffer Sizing**

$$B \approx \sigma \cdot \sqrt{L}$$

- σ: variability of incoming changes
- L: lead time to flush buffer safely

3. **Incident Firebreak**

$$R_t = \frac{C}{T}$$

- R_t: recovery throughput
- C: contained scope
- T: time to reroute + repair

KPIs

- **Lead time for change** (PR opened → deploy live)
- **Change failure rate** (incidents per 100 deploys)
- **MTTR** (mean time to recover service)
- **Buffer health index** (queue length variance ÷ steady-state capacity)
- **Rollout yield** (percent of staged features reaching full prod)

Lab — 12-Period Experiments

Design: One experiment per week/iteration; reset metrics after each cycle.

- **Gate Calibration Sweep**
  Tighten vs. loosen test/approval gates; measure lead time vs. incident rate.
- **Buffer Breathing Test**
  Adjust canary pool size or rollout batch; track queue oscillation and user impact.
- **Firebreak Drill**
  Simulate injected incident (chaos test, DB failover); measure detection latency, reroute speed, MTTR.
- **Memory Refresh**

Run lightweight postmortems and resurface key lessons before the next drill; check if error class recurrence decreases.

Case Card — Staged Feature Rollout

Scenario: A team must ship a new payments module.

- **Gate:** Only 5% of traffic allowed past the feature flag until error budget is <1%.
- **Buffer:** Canary cluster holds feature for 2 days while monitoring stability metrics.
- **Trigger:** Automated rollback script fires if error > X for > 2 minutes.
- **Memory:** Postmortem logged; surfaced in backlog grooming to prevent repeat.

Result: Delivery risk shrinks without slowing throughput; trust in release rhythm grows.

Common Pitfalls

- Over-tightening gates → false bottlenecks, morale collapse.
- Oversized buffers → slow rollout, stale code, hidden debt.
- Firebreak drills skipped → brittle recovery when the real event hits.
- Postmortems written, never resurfaced → memory decays, same mistakes recur.

Ô-peek (one-liner)

Gates, buffers, and firebreaks look like ops plumbing—but in deeper geometry, they are **collapse windows** that define how fast an observer system can re-phase under stress.

Ch.16 Supply Chain & Inventory — Dampers, Reorder Topology, Seal-Bleed Policy

Mechanism

A supply chain is a tension system of **buffers, boundaries, and gates**. Orders flow forward; goods and cash flow backward. Variability at the front propagates as bullwhip oscillation unless damped by buffers and smart gating.

Three primitives dominate:

- 艮兌 **(Boundary & Buffer)** → Inventory buffers, reorder points, safety stock.
- 乾坤 **(Gradient & Gate)** → Seal (hard gate for quality) and bleed (controlled leakage for yield).
- 坎離 **(Memory × Focus)** → Historical demand traces, focus SKUs vs. long tail.

The art is balancing *dampers* (buffers to absorb shocks) with *seal-bleed rules* (decide where to gate hard vs. allow controlled leakage).

Minimal Equations

1. **Safety Stock (Buffer Sizing)**

$$SS \approx z \cdot \sigma_d \cdot \sqrt{L}$$

- z: service level multiplier
- σ_d: demand variability
- L: lead time

2. **Reorder Point**

$$ROP = d \cdot L + SS$$

- d: average demand rate

3. **Seal-Bleed Yield**

$$Y \approx \frac{Q_{sealed} + \beta Q_{bled}}{Q_{total}}$$

- Q_{sealed}: quantity passing strict quality gate
- Q_{bled}: units allowed to pass under relaxed standard
- β: bleed multiplier (discounted yield)

KPIs

- **Fill rate** (fraction of demand met without delay)
- **Cash conversion cycle (CCC)** (days from cash out → cash in)
- **Inventory turnover** (cost of goods ÷ avg. inventory)
- **Bullwhip index** (variance amplification ratio)
- **Seal-bleed yield** (effective output vs. wasted effort)

Lab — 12-Period Experiments

Design: Run experiments across planning cycles (weeks/months).

- **Buffer Breathing Test**
  Adjust safety stock multiplier z; measure bullwhip index and fill rate.
- **Reorder Topology Sweep**
  Shift from periodic review → continuous review; track CCC and service level.
- **Seal-Bleed Stress Test**
  Relax vs. tighten quality thresholds; measure yield, return rate, and trust impact.

- **Memory Refresh**
  Re-inject past demand shocks into forecast models; measure forecast accuracy improvement.

Case Card — Breathing Buffers in Retail Supply

Scenario: A retailer faces seasonal spikes in toy sales.

- **Buffer:** Raise *z* multiplier before holidays; let it "breathe down" post-season.
- **Reorder:** Continuous review of top 10 SKUs; periodic for long-tail SKUs.
- **Seal-Bleed:** Seal safety items (no defect tolerance); bleed fashion items (allow cosmetic defects at discount).
- **Memory:** Feed post-holiday sales traces into next year's forecast.

Result: Stockouts drop, cash cycle shortens, bullwhip dampens.

Common Pitfalls

- Buffers set by gut feel → chronic overstock or shortages.
- Seal applied everywhere → high waste, frozen cash.
- Bleed without control → brand erosion, downstream failures.
- Demand history ignored → repeating the same mis-forecasts.

Ô-peek (one-liner)

Supply chain gates and buffers are not just logistics—they are **semantic attractors** that decide which fluctuations collapse into visible shortages and which vanish into buffers.

Ch.17 Content & Community — Pulse-Soak, Memory Resurfacing, Fatigue Radar

Mechanism

Communities grow on **attention cycles**. Fresh content acts as a **pulse** (short burst of activation); long-tail threads and archives provide **soak** (deep retention). To sustain engagement, you must resurface memory without burning out members.

Three primitives at play:

- 震巽 **(Trigger × Guidance)** → Pulse content, nudge entry routes.
- 坎離 **(Memory × Focus)** → Resurface archives, spotlight contributors.
- 艮兌 **(Boundary & Buffer)** → Fatigue radar, throttle bursts, keep buffers of goodwill.

The system works when pulses ignite without overshoot, soak phases accumulate depth, and

fatigue signals are caught early.

Minimal Equations

1. **Pulse–Soak Engagement**

$$E_t \approx \alpha P_t + \beta S_t$$

- P_t: pulse activity (likes, posts per hour)
- S_t: soak depth (long-thread reads, archive revisits)
- α, β: weighting by community type

2. **Resurfacing Kernel**

$$R(\Delta\tau) = e^{-\lambda \Delta\tau} + \gamma \cdot f(\text{context})$$

- $\Delta\tau$: time since last surfacing
- λ: decay rate
- γ: refresh boost if context matches trend

3. **Fatigue Index**

$$FI = \frac{\text{drop_rate}}{\text{pulse_width}}$$

- Higher FI → burnout risk

KPIs

- **Engagement half-life** (time for pulse activity to halve)
- **Soak ratio** (long-form reads ÷ short-form reactions)
- **Memory resurfacing yield** (views generated from archives)
- **Fatigue index (FI)** (drop rate ÷ pulse width)
- **Retention slope** (weekly active users ÷ monthly active users)

Lab — 12-Period Experiments

- **Pulse Width Test**
  Run events with different durations; measure engagement half-life and FI.
- **Soak Amplifier**
  Highlight long threads in rotation; track soak ratio change.
- **Resurfacing Scheduler**
  Resurface archived posts at varying Δτ; measure yield vs. noise.

- **Fatigue Radar Drill**
  Push extra pulse one week; monitor FI and recovery time.

Case Card — Community Pulse & Soak

Scenario: A developer forum plans a new feature launch.

- **Pulse:** Launch AMA (ask-me-anything) with engineers → spike traffic.
- **Soak:** Archive AMA, pin summary thread for later readers.
- **Resurfacing:** Bring back AMA highlights at next release cycle.
- **Fatigue Radar:** Monitor FI during spike; throttle notifications if drop rate accelerates.

Result: Engagement spike converts into long-tail knowledge base; members stay without overload.

Common Pitfalls

- Only pulsing → boom-bust cycles, fatigue.
- Soak ignored → archives rot, repeat questions multiply.
- Resurfacing too aggressive → "spam" complaints, FI spike.
- Fatigue radar absent → burnout goes undetected until churn.

Ô-peek (one-liner)

Content pulses and soak phases are just scheduling knobs—yet at depth, they are **semantic tick windows** that govern how collective memory collapses into durable culture.

Ch.18 Org & Finance — KPI "Photons" (Reports) as Observables; Cadence Design

Mechanism

Organizations and finance systems run on **signals**. Reports, dashboards, and KPIs are not just paperwork—they are **observables** that collapse uncertainty into action. A report is like a **photon**: it reveals one slice of the system while shaping the response.

Cadence matters as much as content. Weekly standups, monthly closes, quarterly reviews: each is a clock that sets rhythm. Misaligned cadences cause drift, debt, and wasted effort.

Three primitives at play:

- 坎離 **(Memory × Focus)** → Financial records, KPI dashboards, selective focus.
- 乾坤 **(Gradient & Gate)** → Budget gradients, investment gates.
- 艮兌 **(Boundary & Buffer)** → Working capital buffers, accrual vs. cash boundaries.

When observables are crisp, gates clear, and cadences aligned, the org stays coherent instead of spinning out in noise.

Minimal Equations

1. **KPI Photon Signal**

$$I_{obs} = \frac{M}{N}$$

- M: meaningful events captured (e.g., closed deals)
- N: total noise in system (irrelevant transactions)

2. **Cadence Drift**

$$\Delta T = | T_{team} - T_{org} |$$

- T_{team}: team reporting interval
- T_{org}: organizational reporting interval
- Large ΔT → misalignment, wasted sync

3. **Working Capital Buffer**

$$WC = AR + INV - AP$$

- Accounts receivable + Inventory – Accounts payable
- Positive WC = buffer, negative WC = bleed

KPIs

- **Lead-to-close velocity** (sales ops)
- **Days sales outstanding (DSO)** and **days payable outstanding (DPO)**
- **Operating cadence alignment** (variance in reporting intervals)
- **Budget gate precision** (forecast vs. actual variance)
- **Cash buffer days** (operating cushion)

Lab — 12-Period Experiments

- **KPI Photon Calibration**
  Simplify dashboard metrics; measure noise ratio drop.
- **Cadence Alignment Test**
  Shift one team from biweekly → weekly reports; measure ΔT and decision lag.
- **Buffer Stress Drill**
  Reduce WC by X% in sandbox; monitor liquidity shock response.

- **Budget Gate Sweep**
  Tighten vs. loosen approval gates; track throughput vs. leakage.

Case Card — Quarterly Cadence Reset

Scenario: A SaaS org suffers from slow budget approvals and reporting lag.

- **Photon:** Finance consolidates reports into a clean KPI set (ARR, churn, CAC).
- **Gate:** All spend >$50k gated through quarterly review.
- **Buffer:** Raise WC cushion by negotiating DPO extensions.
- **Cadence:** Align product roadmap updates to financial quarter, eliminating ΔT drift.

Result: Decisions accelerate, liquidity risk shrinks, teams stop talking past each other.

Common Pitfalls

- Too many KPIs → dashboards become noise generators.
- Cadence mismatch → finance runs quarterly, product weekly, execs blind to reality.
- Over-gating → every spend stuck, innovation stalls.
- Under-buffering → one shock, and liquidity vanishes.

Ô-peek (one-liner)

Reports and cadences look like admin chores—but in deeper geometry they are **semantic photons**: discrete collapse ticks that give an org its rhythm in time.

Ch.19 The 12-Period Experiment Suite

Mechanism

A playbook without experiments is just theory. The **12-period experiment suite** is the standard way to test, tune, and compare interventions across domains. One "period" = one full cycle of your operating rhythm (week, sprint, month). Twelve periods = enough data to see signal, not just noise.

This suite is not about one-off pilots. It's about **repeatable rhythms**: run, measure, adjust, reset. Each primitive—friction, buffers, guidance, gating, resurfacing—has standard experiments that fit into this 12-period backbone.

Minimal Equations

1. **Noise/Signal Ratio**

$$NSR = \frac{\sigma_{\text{residual}}}{\mu_{\text{effect}}}$$

- High NSR = placebo or randomness.

2. **Fatigue Limit**

$$F_{lim} \approx \frac{\Delta E}{\Delta P}$$

- Drop in engagement ÷ pulse width.

3. **Tick Pacing Error**

$$\Delta \tau = | \tau_{exp} - \tau_{org} |$$

- Experiment cadence vs. org cadence.

Standard Labs

1. **Friction Sweep**

- Vary entry/exit steps (e.g., clicks, approvals).
- KPIs: throughput, abandonment, cycle time.

2. **Buffer Breathing**

- Expand/contract buffers (inventory, canary size).
- KPIs: oscillation amplitude, backlog half-life.

3. **Guidance Intensity**

- Change nudges or routing strength.
- KPIs: conversion, fatigue index.

4. **Gate Calibration**

- Adjust thresholds (test coverage %, budget gates).
- KPIs: quality pass rate, leakage yield.

5. **Resurfacing Rhythm**

- Rotate old content, cases, or lessons into view.
- KPIs: resurfacing yield, recall latency.

Measuring Integrity

- **Noise Checks:** Always hold a control group; compare NSR.
- **Placebo Controls:** Run “sham” nudges or false deadlines to measure background effect.

- **Fatigue Limits:** Monitor FI each period; reset if slope spikes.
- **Tick Pacing:** Align period length to real cadence (sprint, month, season). Misalignment = false conclusions.

Tools & Templates

- **Colab notebooks**: ready Python templates for effect size, NSR, and FI plots.
- **Excel dashboards**: 12-period KPI tracker with pivot slices.
- **CSV schema**:

  ```
  period, intervention, cohort, metric_name, metric_value, notes
  ```

  Standardized so teams can aggregate results or plug into BI.

Case Card — Guidance Intensity Ramp

Scenario: An onboarding funnel shows steep drop-offs.

- Periods 1–3: baseline (no extra nudges).
- Periods 4–6: light nudges (one reminder).
- Periods 7–9: medium nudges (reminder + route).
- Periods 10–12: heavy nudges (auto-prompt + incentive).

Result: Conversion rises at medium; drops at heavy (fatigue spike). Sweet spot = guidance intensity 2/3.

Common Pitfalls

- Too short (<6 periods) → conclusions = noise.
- No placebo → any “improvement” may be coincidence.
- Ignoring fatigue → interventions look good until they burn users.
- Periods misaligned with org cadence → results don’t transfer.

Ô-peek (one-liner)

Twelve periods aren’t magic—they’re **collapse ticks** that force rhythm into learning; cadence is the real variable under test.

Ch.20 Metrics, Alerting, and Saturation Hygiene

Mechanism

Metrics are your **org's sense organs**. They tell you what's moving, what's stuck, and when you're about to hit collapse. But metrics alone aren't enough—you need alerting (when thresholds are breached) and hygiene (so they don't ossify into noise).

Saturation is the silent killer. A metric that once drove growth can turn into a **semantic black hole**: everything collapses into it, yet nothing new emerges. Hygiene practices keep metrics fresh, prevent KPI sprawl, and fight ossification.

KPI Catalog

A few canonical measures, with definitions, units, and baselines:

KPI	Definition	Unit	Typical Baseline	Notes
Throughput	Output per unit time	items/ day	Historical avg	Core flow measure
Lead Time	Entry → delivery	days	P50 / P90 split	Watch for tail risk
Fill Rate	Demand met without delay	%	>95%	Supply chain, inventory
Retention Slope	Week N / Month N users	ratio	0.2–0.4 (consumer apps)	Shape tells memory health
Fatigue Index (FI)	Drop rate ÷ pulse width	scalar	<0.1	>0.2 = burnout risk
Cash Buffer Days	Operating liquidity cushion	days	30–60	Less → fragile, more → idle
MTTR	Mean time to recovery	hours	<1 for SaaS infra	Reliability heartbeat
CCC	Cash conversion cycle	days	Sector dependent	Shorter = healthier

Baseline values vary by domain—use them only as starting anchors.

Saturation & Black-Hole Diagnostics

Red flags that a metric has collapsed into a black hole:

- **Endless repetition:** KPI flat-lines but teams keep staring at it.
- **Misaligned effort:** Initiatives optimize the metric but hurt real outcomes.
- **Attractor lock-in:** Budget, careers, and dashboards all orbit one number.
- **Blindness:** Other signals ignored ("but our DAU is up!").

Diagnostics:

- Run **entropy test**: variance across KPIs; if one dominates >80%, collapse risk.
- Run **tick-lag test**: does the metric move only after long delays? If yes, you're watching an afterimage.
- Run **observer divergence test**: do different teams interpret the same number differently? If yes, semantic decoherence.

Anti-Ossification Plays

- **Metric Rotation:** Swap 10–20% of dashboard KPIs every quarter.
- **Composite Refresh:** Recalculate indices with fresh weights annually.
- **Counter-Metrics:** Pair each KPI with its failure twin (e.g., throughput vs. error rate).
- **Drill-Back Rituals:** Once per cycle, revisit raw logs/data to check that KPIs still map to ground truth.

Alerting Framework

- **Thresholds:** Simple rules (MTTR > 1 hr, FI > 0.2).
- **Rate-of-change triggers:** Alerts when slope exceeds baseline deviation.
- **Saturation alarms:** Alert when variance/entropy ratio falls below threshold.
- **Cadence check:** Auto-flag metrics that update slower than org cadence.

Case Card — Anti-DAU Black Hole

Scenario: A consumer app is obsessed with DAU. Engagement plateaus; retention falls.

- **Diagnostic:** DAU variance dominates >85% of dashboard entropy.
- **Play:** Rotate in retention slope + fatigue index; demote DAU from #1 slot.
- **Result:** Teams start optimizing for durability, not just daily clicks.

Common Pitfalls

- KPI sprawl: too many metrics, none trusted.
- Metric ossification: numbers survive long after they stop mattering.
- Black-hole worship: whole org bends around a stale attractor.
- Alert spam: false positives erode trust.

Ô-peek (one-liner)

Metrics are more than numbers—they are **collapse traces**; saturation entropy shows when an observer system has stopped learning.

Appendix A — Trigram ↔ Engineering Primitive Map

Eight Incubation Trigram (先天八卦) primitives, mapped to **engineering mechanisms**. Use this as your 1-page reference when designing flows, experiments, or dashboards.

Trigram(卦)	Classical Pairing	Engineering Primitive	Mechanism Analogy	Default KPIs	Failure Smells

Trigram(卦)	Classical Pairing	Engineering Primitive	Mechanism Analogy	Default KPIs	Failure Smells
乾 **(Heaven)**	Source / Father	**Potential Gradient**	Capacity → demand slope	Throughput, lead time	Starvation, overload
坤 **(Earth)**	Sink / Mother	**Gating Surface**	Orifice, valve, qualify vs. reject	Abandonment rate, yield	Leakage, false blocks
艮 **(Mountain)**	Stillness / Stop	**Boundary**	Hard stop, constraint wall	Lead time variance	Frozen queues, bottlenecks
兌 **(Marsh)**	Joy / Exchange	**Buffer / Exchange**	Inventory, dampers, breathing buffers	Fill rate, WIP	Bullwhip oscillation, dead stock
震 **(Thunder)**	Shock / Trigger	**Trigger**	Event ignition, nudge, spark	Activation %, response time	Missed triggers, fatigue
巽 **(Wind)**	Penetrating / Flow	**Guidance**	Routing, steering, nudges along path	Route efficiency, step-drop	Misroutes, user churn
坎 **(Water)**	Pit / Risk	**Memory**	Retention kernel, decay curve, recall store	Retention slope, recall latency	Forgetting, silent churn
離 **(Fire)**	Bright / Clarity	**Focus**	Spotlight, attention filter, priority lens	Focus ratio, time-on-task	Distraction, scattered effort

---

Usage Notes

- **Dyads:** Pair two primitives for core labs (e.g., 乾×坤 = Gradient & Gate → throughput control).
- **Modes:** Combine dyads for system patterns (e.g., Pulse–Soak = 震巽 + 坎).
- **Triads:** Add a stabilizer or memory element to compound effects.
- **Four-in-One:** All eight → complete operating diagram.

Ô-peek (one-liner)

Each primitive is more than a knob—it is a **phase attractor**. In Version B, you'll see how these map into collapse geometry.

Appendix B — KPI & Equation Cheats

(ready to paste into notebooks, spreadsheets, or dashboards)

Flow & Friction

- **Throughput (Q):**

$$Q \approx \alpha \cdot \Delta V \cdot f(\text{fit}) \cdot (1 - \mu)$$

• ΔV = capacity–demand gap
• μ = friction coefficient
• f(fit) = alignment factor

- **Lead Time (LT):**

$$LT = \frac{WIP}{Q}$$

- **Abandonment Rate:**

$$AR = \frac{\text{dropouts}}{\text{entries}}$$

Buffers & Boundaries

- **Safety Stock (SS):**

$$SS \approx z \cdot \sigma_d \cdot \sqrt{L}$$

- **Reorder Point (ROP):**

$$ROP = d \cdot L + SS$$

- **Bullwhip Index (BI):**

$$BI = \frac{\sigma_{\text{orders}}}{\sigma_{\text{demand}}}$$

Triggers & Guidance

- **Activation Energy (Ea):**

$$P(\text{activation}) \propto e^{-Ea\,/\,kT}$$

(probability rises as activation energy falls)

- **Route Efficiency (RE):**

$$RE = \frac{\text{optimal steps}}{\text{actual steps}}$$

- **Step Drop (SD):**

$$SD = \frac{\text{dropouts at step}}{\text{entries at step}}$$

Memory & Focus

- **Retention Kernel:**

$$R(t) = R_0 \cdot e^{-\lambda t} + \gamma \cdot f(\text{refresh})$$

- **Recall Latency (RL):**

$$RL = t_{\text{recall}} - t_{\text{exposure}}$$

- **Focus Ratio (FR):**

$$FR = \frac{\text{time on focal tasks}}{\text{total time}}$$

Reliability & Recovery

- **Change Failure Rate (CFR):**

$$CFR = \frac{\text{failed changes}}{\text{total changes}}$$

- **MTTR (Mean Time to Recover):**

$$MTTR = \frac{\sum \text{recovery times}}{\text{incident count}}$$

- **Recovery Throughput (Rt):**

$$R_t = \frac{C}{T}$$

(scope contained ÷ time to reroute+repair)

Saturation & Fatigue

- **Noise–Signal Ratio (NSR):**

$$NSR = \frac{\sigma_{\text{residual}}}{\mu_{\text{effect}}}$$

- **Fatigue Index (FI):**

$$FI = \frac{\text{drop rate}}{\text{pulse width}}$$

- **Entropy Test (ET):**

$$ET = \frac{\max(KPI\ variance)}{\sum (KPI\ variance)}$$

(>0.8 = black-hole risk)

Quick Baselines

- Lead time: P50/P90 split
- Fill rate: ≥95%
- Retention slope: 0.2–0.4 (consumer apps)

- FI: <0.1 safe, >0.2 burnout risk
- Cash buffer days: 30–60

Ô-peek (one-liner)

Equations are not just math—they are **collapse operators** that decide which tensions show up as reality.

Appendix C — Case Card Library

Software Delivery

Case: Canary Rollout with Firebreak Drill

- **Setup:** Payments feature ready, 2M users.
- **Intervention:** Roll out to 5% canary cluster, monitor error <1%. Inject chaos test (DB failover).
- **Observation:** MTTR logged, queue length variance tracked.
- **Result:** Stable at 5% → proceed; chaos drill proves rollback works in <2m.

Supply Chain

Case: Breathing Buffers for Seasonal Spike

- **Setup:** Retailer prepares for holiday toy demand.
- **Intervention:** Raise safety stock multiplier z in periods 1–3; lower back post-holiday.
- **Observation:** Fill rate vs. bullwhip index.
- **Result:** Stockouts ↓, cash cycle stable.

Content & Community

Case: Pulse–Soak AMA

- **Setup:** Dev forum wants more sustained engagement.
- **Intervention:** Pulse = live AMA; Soak = archive & pin summary thread.
- **Observation:** Engagement half-life, soak ratio.
- **Result:** Spike converts into durable knowledge base.

Org & Finance

Case: Quarterly Cadence Reset

- **Setup:** SaaS firm has budget lag + mismatched cadences.
- **Intervention:** Photon = clean KPI set (ARR, churn, CAC). Align roadmap updates to quarter.
- **Observation:** ΔT drift, cash buffer days.
- **Result:** Decision lag ↓, liquidity risk ↓, alignment ↑.

Reliability & Incidents

Case: Rotating Firebreak Weak Link

- **Setup:** Infra team rehearses incident response.
- **Intervention:** Randomly disable a non-core node each week.
- **Observation:** Containment time, spill cost.
- **Result:** MTTR drops by 40% over 12 periods.

Growth Funnel

Case: Guidance Intensity Ramp

- **Setup:** B2B onboarding funnel with steep drop-off.
- **Intervention:** Period 1–3 = no nudges, 4–6 = light, 7–9 = medium, 10–12 = heavy.
- **Observation:** Conversion vs. FI.
- **Result:** Sweet spot at medium intensity; heavy triggers fatigue spike.

Inventory Policy

Case: Seal vs. Bleed Decision

- **Setup:** Manufacturer facing minor cosmetic defects.
- **Intervention:** Seal = block; Bleed = sell discounted.
- **Observation:** Effective yield, return rate.
- **Result:** Bleed policy improves yield 15% with no brand damage.

Learning & Retention

Case: Memory Resurfacing Scheduler

- **Setup:** E-learning platform sees students forgetting modules.
- **Intervention:** Resurface lessons at $\Delta\tau$ = 7d, 14d, 30d.
- **Observation:** Recall latency, retention slope.

- **Result:** 14d resurfacing gives best balance of recall vs. fatigue.

Org Culture

Case: Anti-DAU Black Hole

- **Setup:** Consumer app dashboard dominated by DAU.
- **Intervention:** Rotate in retention slope + FI, demote DAU.
- **Observation:** Dashboard entropy (ET).
- **Result:** Teams shift to long-term health; churn stabilizes.

Cash Flow

Case: Working Capital Stress Drill

- **Setup:** Mid-size manufacturer with tight cash.
- **Intervention:** Simulate 20% AR delay; track WC.
- **Observation:** Cash buffer days, CCC.
- **Result:** Exposes liquidity gap → negotiate supplier DPO extension.

General Note

Each case card is **short, runnable, measurable**. Plug into the **12-period suite** (Ch.19), track KPIs, compare against baselines.

Ô-peek (one-liner)

Case cards are more than exercises—they are **observer traces**, showing where collapse choices actually leave marks.

Appendix D — Ô-peek Cross-Reference

This appendix collects all **Ô-peek callouts** from Version A and points to their **expanded treatment** in Version B. Use it as a map: what looks like a half-line teaser in this book will open into full geometry in the companion volume.

Part I — Dyads

- **Ch.2 Gradient & Gate (乾坤):**
  Ô-peek → *observer frame reallocates collapse odds across channels*
  ↪ **Book 2:** Projection Operator Ô, probability amplitudes $P_j(\tau)$, collapse frame shifts.

- **Ch.3 Boundary × Buffer (艮兌):**

Ô-peek → *phase interchange across boundaries (山澤通氣)*
↳ **Book 2:** Phase alignment mechanics, boundary permeability, semantic energy interchange.

- **Ch.4 Trigger × Guidance (震巽):**
  Ô-peek → *phase-lock vs. tick desynchrony drives fatigue*
  ↳ **Book 2:** Semantic clocks, synchronization failure, fatigue as collapse drift.

- **Ch.5 Memory × Focus (坎離):**
  Ô-peek → *near-linear behavior inside semantic BH zones enables stable control*
  ↳ **Book 2:** Nonlinear semantic wavefunctions, black-hole near-linearity, control in saturation zones.

Part II — Two-Dyad Modes

- **Ch.6 Ventilate–Store:**
  Ô-peek → *phase interchange + tick pacing*
  ↳ **Book 2:** Oscillatory modes, τ-cycle alignment, boundary–memory coupling.

- **Ch.7 Ignite–Guide:**
  Ô-peek → *phase-lock windows*
  ↳ **Book 2:** Lock-in phenomena, semantic entrainment, campaign ignition geometry.

- **Ch.8 Seal–Bleed:**
  Ô-peek → *observer frame changes "what counts" as qualified*
  ↳ **Book 2:** Measurement relativity, collapse boundary conditions, observer redefinition.

- **Ch.9 Pulse–Soak:**
  Ô-peek → *latent iT buildup before ticks*
  ↳ **Book 2:** Imaginary time *iT*, latent tension accumulation, collapse triggering.

Part III — Triads

- **Ch.10 Compounding Trio:**
  Ô-peek → *τ-cycle alignment across subsystems*
  ↳ **Book 2:** Multi-system synchrony, attractor compounding, hysteresis traces.

- **Ch.11 Crisis Trio:**
  Ô-peek → *collapse entropy spikes warn of saturation*
  ↳ **Book 2:** Entropy measures, collapse thresholds, systemic firebreaks.

- **Ch.12 Growth Flywheel:**
  Ô-peek → *attractor formation measurable as phase curvature*
  ↳ **Book 2:** Semantic curvature, attractor basin geometry, phase-coherent growth.

Part IV — Eight-Node Diagram

- **Ch.13 Eight-Node Control Board:**
  Ô-peek → *eight attractors as a semantic OS*
  ↳ **Book 2:** Semantic operating systems, observer–node mapping, Trigram as attractor lattice.

- **Ch.14 Synchronization, Drift, Debt:**
  Ô-peek → *semantic clocks and observer-bound evolution*
  ↪ **Book 2:** Collapse delay, drift geometry, organizational time dilation.

Part V — Domain Playbooks

- **Ch.15 Software Delivery:**
  Ô-peek → *collapse windows under stress*
  ↪ **Book 2:** Stress-field collapse, observer re-phasing, resilience geometry.
- **Ch.16 Supply Chain & Inventory:**
  Ô-peek → *semantic attractors deciding fluctuation collapse*
  ↪ **Book 2:** Buffer attractor math, phase filtering, fluctuation geometry.
- **Ch.17 Content & Community:**
  Ô-peek → *semantic tick windows for collective memory*
  ↪ **Book 2:** Memory collapse ticks, community attractors, cultural time sync.
- **Ch.18 Org & Finance:**
  Ô-peek → *semantic photons as discrete collapse ticks*
  ↪ **Book 2:** KPI as observables, photon analogy, semantic Planck units.

Part VI — Lab Handbook

- **Ch.19 Experiment Suite:**
  Ô-peek → *twelve collapse ticks as learning cadence*
  ↪ **Book 2:** Experiment as observer–tick alignment, τ-period resonance.
- **Ch.20 Metrics & Hygiene:**
  Ô-peek → *metrics as collapse traces; entropy as stop-learning signal*
  ↪ **Book 2:** Collapse entropy formalism, black-hole attractors, observer bias in measurement.

Quick Index

- **Ô:** Projection operators, observer frames (Ch.2, Ch.8, Ch.13).
- **τ (semantic time):** Tick pacing, synchrony, fatigue drift (Ch.4, Ch.6, Ch.10, Ch.19).
- **Phase alignment:** Curvature, lock-in, hysteresis (Ch.3, Ch.7, Ch.12).
- **Semantic black holes:** Saturation, near-linear control, entropy spikes (Ch.5, Ch.11, Ch.20).

Closing Note

Ô-peeks in this book are **breadcrumbs**. They keep Version A practical, while preparing readers for the deeper semantic field logic of Version B.

Appendix E — Glossary & Further Reading

Glossary (Practical Definitions)

- **Incrubation Trigram (先天八卦):** Eight classical symbols mapped here to engineering primitives (gradient, gate, buffer, etc.). Think of them as "the eight knobs" of any system.
- **Dyad:** Pair of primitives used together (e.g., Gradient × Gate). Basic building block of playbooks.
- **Triad:** Three-primitive kit for compounding effects (e.g., Gradient + Retention + Buffer).
- **Four-in-One:** All eight primitives integrated into one operating diagram—complete system view.
- **Gate:** A rule or filter that qualifies what passes (features, budget, inventory).
- **Buffer:** A reservoir that smooths variability (inventory, queues, attention).
- **Trigger:** Event or condition that initiates flow (onboarding nudge, incident alert).
- **Guidance:** Steering mechanism that routes flow along paths (nudges, recommendations).
- **Memory:** Retained trace of past events (user retention, financial records).
- **Focus:** Spotlight on what matters most now (attention, priority tasks).
- **Friction:** Resistance to flow, intentional or accidental (extra clicks, approvals, bottlenecks).
- **Fatigue Index (FI):** Drop-rate ÷ pulse width; early warning for burnout in users or systems.
- **KPI Photon:** A single observable metric/report that collapses uncertainty into action.
- **Saturation:** When a KPI or process has stopped yielding new insight—flatline, ossified.

Further Reading (Practical First Steps)

- **Lean & Flow:**
  The Goal (Eliyahu Goldratt) — classic on constraints and throughput.
  Lean Thinking (Womack & Jones) — buffer and waste reduction in practice.
- **Operations & Reliability:**
  Site Reliability Engineering (Google SRE book) — playbook for gating, buffers, incident drills.
  The Phoenix Project (Kim, Behr, Spafford) — narrative on flow and friction in software delivery.
- **Measurement & Metrics:**
  How to Measure Anything (Douglas Hubbard) — turning intangibles into observables.
  Lean Analytics (Croll & Yoskovitz) — KPI design and iteration.
- **Community & Engagement:**
  Building Successful Online Communities (Kraut & Resnick) — guidance, pulse, and soak cycles.

For Later (Deferred to Book 2)

If you want to explore the deeper layer (Ô, τ, semantic fields, black holes), hold until Version B. The callouts in this book already map to those chapters.

Ô-peek (one-liner)

This glossary is pragmatic scaffolding; in Version B, every term re-expands into **semantic geometry**.

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.